

INFORMATIQUE • 3ÈME

INFORMATIQUE – CLASSE DE 3ÈME

LEÇON N15

UTILISATION DES CONCEPTS DE
BASE DU DÉVELOPPEMENT
LOGICIEL

DURÉE CONSEILLÉE : 1 SÉANCE (FICHE DE PROGRESSION HARMONISÉE)

DANS CETTE LEÇON

Cette leçon initie aux concepts fondamentaux du développement logiciel : cycle de vie, algorithmique, structures de contrôle et modularité. Elle permet de structurer une solution informatique pour résoudre un problème concret du quotidien.

- À la fin de cette leçon, je sais décrire les grandes étapes du cycle de vie d'un logiciel (analyse, conception, codage, test, maintenance).
- À la fin de cette leçon, je sais distinguer un algorithme (logique) d'un programme (code source) et expliquer le rôle du compilateur ou de l'interpréteur.
- À la fin de cette leçon, je sais identifier et utiliser les trois structures de contrôle fondamentales : séquence, sélection (SI) et itération (TANT QUE / POUR).
- À la fin de cette leçon, je sais déclarer des variables adaptées (entier, réel, chaîne, booléen) et leur affecter des valeurs.
- À la fin de cette leçon, je sais décomposer un problème en sous-problèmes en utilisant des fonctions ou procédures (modularité).
- À la fin de cette leçon, je sais rédiger un algorithme en pseudo-code standard (style Bac/BEPC) pour résoudre un problème simple de la vie courante.

TÉLÉCHARGE
ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. 1. Le cycle de vie d'un logiciel : de l'idée au produit
2. 2. De l'algorithme au programme : langages et traduction
3. 3. Les briques de base : Variables, Types et Structures de contrôle
4. 4. Modularité : Fonctions et Procédures (Diviser pour régner)
5. 5. Valider et documenter : Tests, Débogage et Qualité
6. 6. Activité d'intégration
7. 7. Méthodes et exercices résolus
8. 8. Exercices
9. 9. Corrigés des exercices
10. 10. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais décrire les grandes étapes du cycle de vie d'un logiciel (analyse, conception, codage, test, maintenance).
- À la fin de cette leçon, je sais distinguer un algorithme (logique) d'un programme (code source) et expliquer le rôle du compilateur ou de l'interpréteur.
- À la fin de cette leçon, je sais identifier et utiliser les trois structures de contrôle fondamentales : séquence, sélection (SI) et itération (TANT QUE / POUR).
- À la fin de cette leçon, je sais déclarer des variables adaptées (entier, réel, chaîne, booléen) et leur affecter des valeurs.
- À la fin de cette leçon, je sais décomposer un problème en sous-problèmes en utilisant des fonctions ou procédures (modularité).
- À la fin de cette leçon, je sais rédiger un algorithme en pseudo-code standard (style Bac/BEPC) pour résoudre un problème simple de la vie courante.
- À la fin de cette leçon, je sais construire un jeu de tests (valeurs normales, limites, invalides) pour valider un algorithme et justifier mes choix.
- À la fin de cette leçon, je sais expliquer la différence entre erreur de syntaxe, erreur d'exécution et erreur logique (bug).

PRÉREQUIS

- Notion d'algorithme et de pseudo-code (variables, affichage, lecture) vue en 4ème.
- Utilisation basique d'un éditeur de texte et navigation dans les dossiers/fichiers.

- Logique booléenne simple (VRAI/FAUX, ET, OU, NON) abordée en mathématiques.
- Résolution de problèmes arithmétiques simples (opérations, pourcentages).

1 LE CYCLE DE VIE D'UN LOGICIEL : DE L'IDÉE AU PRODUIT

À Yaoundé, Mme Ngo gère une tontine (Njangi) de 20 membres. Chaque semaine, elle note manuellement les cotisations dans un cahier usé, calcule les intérêts à la main, détermine l'ordre des tours et prépare les reçus. Cette tâche lui prend des heures, la fatigue aidant, des erreurs s'y glissent : un montant mal reporté, un tour oublié pour Paul, un intérêt mal calculé pour Marie. Un jeune développeur, cousin d'un membre, lui propose de créer une application simple pour automatiser ce processus. Mme Ngo accepte, soulagée. Mais comment passer du cahier de comptes à un programme fiable ? Quelles sont les étapes indispensables pour garantir que l'application ne perdra pas l'argent de la tontine et respectera les règles tacites du groupe ? C'est tout l'objet du **cycle de vie d'un logiciel** : une démarche structurée, rigoureuse et itérative pour transformer un besoin réel en un outil numérique robuste.

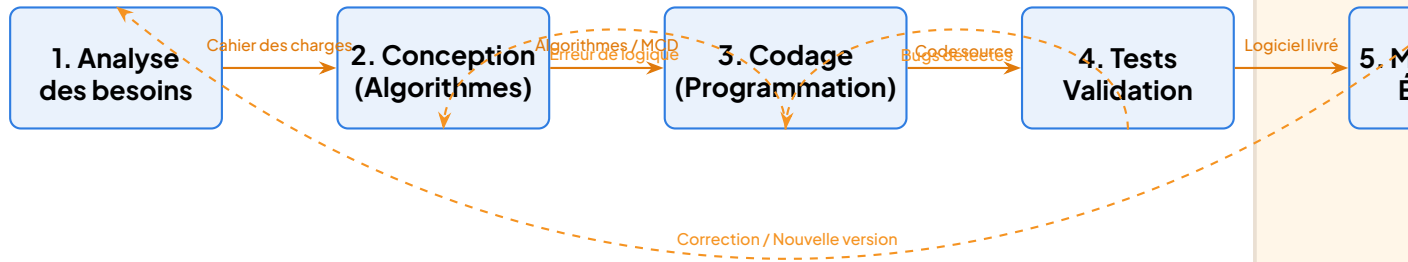
DÉFINITION Logiciel

Un **logiciel** est l'ensemble des programmes, procédures, règles et documentation relatifs au fonctionnement d'un système de traitement de l'information. Contrairement au matériel (hardware) que l'on peut toucher, le logiciel est immatériel : c'est la connaissance figée dans du code.

Le **développement logiciel** n'est pas une simple séance de frappe au clavier. C'est un processus d'ingénierie complet qui va de la naissance de l'idée jusqu'au retrait de l'application (maintenance, évolution, retrait). Pour ne rien oublier — ni le tour de Paul, ni l'intérêt de Marie —, les ingénieurs suivent des modèles de cycle de vie. Le plus classique, et le plus adapté à notre niveau d'initiation, est le **modèle en cascade (ou cycle en V simplifié)**.

1.1 Le modèle en cascade : une vue d'ensemble structurée

Imaginez une cascade : l'eau coule d'un bassin à l'autre, du haut vers le bas. Dans le modèle en cascade, chaque phase doit être terminée (et validée) avant de passer à la suivante. Si l'on découvre une erreur tardivement, il faut « remonter » la cascade, ce qui est coûteux. C'est pourquoi la rigueur en amont (analyse, conception) est vitale.



Le cycle de vie en cascade simplifié : flux descendant (flèches pleines) et boucles de rétroaction (flèches pointillées orange) pour correction et évolution.

À RETENIR**Les 5 phases du cycle de vie (Modèle cascade simplifié)**

1. Analyse des besoins : Comprendre quoi faire et pourquoi. Résultat : Cahier des charges.
2. Conception : Déterminer comment faire. Rédaction des algorithmes (pseudo-code), modélisation des données (MCD). Indépendant du langage.
3. Codage (Implémentation) : Traduire l'algorithme dans un langage de programmation (C, Python, JavaScript...). Résultat : Fichiers sources.
4. Tests (Validation) : Vérifier que le programme fait exactement ce qui est demandé. Tests unitaires, tests d'intégration, tests utilisateur.
5. Maintenance : Corriger les bugs découverts plus tard, adapter aux nouvelles règles (ex : changement de taux d'intérêt), ajouter des fonctionnalités.

1.2 Phase 1 : Analyse des besoins — Comprendre le problème réel

C'est l'étape la plus critique. « Un problème bien posé est à moitié résolu » (John Dewey). Pour la tontine de Mme Ngo, il ne suffit pas de dire « je veux une appli ». Il faut formaliser les règles métier, souvent implicites.

MÉTHODE**Analyser un besoin : la démarche en 3 étapes**

1. Identifier les données d'entrée (Entrées) : Quelles informations le système reçoit-il ?
 - Montant de la cotisation hebdomadaire (ex : 5 000 FCFA).
 - Nombre de membres (ex : 20).
 - Liste des membres (Nom, Téléphone, Rang dans le tour).

- Taux d'intérêt appliqué aux retards (ex : 5% par semaine de retard).
 - Date de début de la tontine.
2. Identifier les résultats attendus (Sorties) : Que doit produire le système ?
- Planning des tours (Qui reçoit la caisse cette semaine ?).
 - État de la caisse totale (Somme des cotisations + intérêts perçus).
 - Suivi des retards par membre (Nombre de semaines, Montant dû).
 - Reçus imprimables ou SMS de confirmation.
3. Déterminer les traitements (Transformations) : Quels calculs, tris, décisions transforment les entrées en sorties ?
- Calcul du tour : Rotation circulaire sur la liste des membres.
 - Calcul des intérêts : Si cotisation payée après le vendredi → Pénalité = Cotisation × Taux × Semaines de retard.
 - Mise à jour caisse : Caisse = Caisse + Cotisations + Pénalités - Montant versé au bénéficiaire du tour.
 - Gestion des exceptions : Membre décédé, membre exclu, entrée d'un nouveau membre en cours de cycle.

EXEMPLE Application à la tontine « Espoir » de Mme Ngo

Le cahier des charges issu de l'analyse pourrait contenir cette règle métier :

Règle R1 (Calcul pénalité) : Si un membre verse sa cotisation avec n semaines de retard ($n \geq 1$), il doit payer une pénalité $P = C \times t \times n$, où C est la cotisation hebdomadaire (5 000 FCFA) et t le taux d'intérêt (0,05 soit 5%).

Cette règle est claire, non ambiguë, chiffrée. Elle servira de base aux tests plus tard.

1.3 Phase 2 : Conception — L'algorithme, langage universel

Une fois le quoi connu, on définit le comment logique, sans encore choisir C, Python ou JavaScript. On écrit un **algorithme**.

DÉFINITION Algorithme

Un **algorithme** est une suite finie et non ambiguë d'instructions élémentaires permettant de résoudre un problème ou d'obtenir un résultat à partir de données d'entrée. Il s'exprime en pseudo-code (français structuré, anglais courant) compréhensible par tout in-

formaticien, indépendamment de la machine ou du langage.

ATTENTION Algorithme ≠ Programme

Ne confondez jamais les deux :

- L'algorithme décrit la logique (la recette de cuisine). Il se lit par l'humain, il est universel, il n'a pas de syntaxe stricte imposée par un compilateur.
- Le programme est la traduction de l'algorithme dans un langage de programmation précis (C, Python, JavaScript...). Il obéit à une syntaxe stricte (points-virgules, accolades, majuscules/minuscules). Il s'exécute par la machine.

Piège fréquent : Écrire du pseudo-code avec la syntaxe du C (ex : `for(i=0;i<n;i++)`) en pensant faire de l'algorithmique. Non ! En algorithmique, on écrit : Pour *i* de 0 à *n*-1 faire.

Voici la traduction de la Règle R1 en algorithmique standard (style français, proche de l'écriture au BEPC).

```

Algorithme CalculPenalite
// But : Calculer la pénalité de retard pour un membre de la tontine
// Entrées : cotisation (Réel), tauxInteret (Réel), semainesRetard (Entier)
// Sortie : penalite (Réel)

Variables
    cotisation, tauxInteret, penalite : Réel
    semainesRetard : Entier
Début
    Afficher "Entrez la cotisation hebdomadaire (FCFA) : "
    Saisir cotisation
    Afficher "Entrez le taux d'intérêt (ex : 0.05 pour 5%) : "
    Saisir tauxInteret
    Afficher "Entrez le nombre de semaines de retard : "
    Saisir semainesRetard

    Si semainesRetard > 0 Alors
        penalite <- cotisation * tauxInteret * semainesRetard
        Afficher "Pénalité à payer : ", penalite, " FCFA"
    Sinon
        Afficher "Aucun retard, pas de pénalité."
        penalite <- 0
    FinSi
Fin
  
```

Listing 1 – Algorithme : CalculPenalite

Remarquez la clarté : Si ... Alors ... Sinon ... FinSi, <- pour l'affectation, pas de ; ni d'accolades. C'est lisible par un gestionnaire de tontine non informaticien.

1.4 Phase 3 : Codage — Traduction dans un langage de programmation

C'est le passage du « langage humain structuré » au « langage machine compréhensible par compilateur/interpréteur ». On choisit un langage adapté : C pour la performance/système, Python pour la rapidité de script, JavaScript pour le web.

Le même algorithme devient un programme C compilable. La rigueur syntaxique est obligatoire.

```
#include <stdio.h> // Bibliothèque standard pour entrée/sortie
#include <stdlib.h> // Pour EXIT_SUCCESS

int main() {
    // 1. Déclaration des variables (Typage fort en C)
    float cotisation, tauxInteret, penalite;
    int semainesRetard;

    // 2. Saisie des données (Entrées)
    printf("Entrez la cotisation hebdomadaire (FCFA) : ");
    scanf("%f", &cotisation); // & obligatoire pour l'adresse mémoire

    printf("Entrez le taux d'interet (ex : 0.05 pour 5%%) : ");
    scanf("%f", &tauxInteret);

    printf("Entrez le nombre de semaines de retard : ");
    scanf("%d", &semainesRetard);

    // 3. Traitement (Structure de contrôle Si/Else)
    if (semainesRetard > 0) {
        penalite = cotisation * tauxInteret * semainesRetard;
        printf("Penalite a payer : %.2f FCFA\n", penalite);
    } else {
        printf("Aucun retard, pas de penalite.\n");
        penalite = 0.0f;
    }

    return EXIT_SUCCESS; // Fin normale du programme
}
```

Listing 2 – Programme C : calcul_penalite.c

PROPRIÉTÉ Compilation vs Interprétation

Une fois le code source écrit (fichier .c, .py, .js), il doit devenir exécutable.

- Langage compilé (C, C++) : Un **compilateur** (ex : gcc) traduit tout le code source en code machine (binaire, fichier .exe ou .out) avant l'exécution. Erreurs = Erreurs de compilation (syntaxe). Exécution = Rapide.
- Langage interprété (Python, JavaScript) : Un **interpréteur** lit, traduit et exécute ligne par ligne à la volée. Pas de fichier exécutable intermédiaire. Erreurs = Erreurs d'exécution (runtime). Développement = Plus interactif.

Pour notre tontine, un petit script Python ou une page web HTML/JS suffit amplement.

1.5 Phases 4 et 5 : Tests et Maintenance — La qualité dans la durée

Un programme qui compile (ou s'exécute sans planter) n'est pas un programme **correct**. Il doit faire exactement ce que le cahier des charges demande.

Les Tests (Validation).

- **Tests unitaires** : On teste chaque fonction isolément. Ex : Appeler `CalculPenalite(5000, 0.05, 2)` → Attendu : 500 FCFA.
- **Tests d'intégration** : On teste l'enchaînement : Saisie → Calcul → Mise à jour caisse → Affichage reçu.
- **Tests utilisateur (Recette)** : Mme Ngo utilise l'appli sur une vraie semaine de tontine. Elle valide : « Oui, c'est bien le tour de Paul, l'intérêt de Marie est juste. »

La Maintenance. Le logiciel vit. Au bout de 6 mois, la tontine décide : « Le taux passe à 10% » ou « On ajoute une cotisation exceptionnelle pour la fête de fin d'année ».

- **Maintenance corrective** : Corriger un bug (ex : l'appli plantait si un membre payait 0 FCFA).
- **Maintenance adaptative** : Adapter à l'environnement (ex : changement de format de date, nouvelle version d'Android).
- **Maintenance évolutive** : Ajouter des fonctionnalités (ex : envoi SMS automatique via API Orange Money).

C'est pourquoi le code doit être **lisible**, **commenté** et **modulaire** (voir section 4 sur les fonctions).

EXERCICE RÉSOLU 1

Test unitaire de la fonction de pénalité

Énoncé : On dispose de la fonction C suivante :

```
float calculerPenalite(float cotisation, float taux, int semaines) {  
    if (semaines > 0) return cotisation * taux * semaines;  
    return 0.0f;  
}
```

Proposer un jeu de tests unitaires complet (cas nominaux, cas limites, cas d'erreur) sous forme de tableau.

Solution détaillée : Un bon jeu de tests couvre les branches du `if` et les valeurs extrêmes.

Cas de test	Entrées (coti- sation, taux, se- maines)	Résultat at- tendu	Résultat ob- tenu	Statut
Cas nominal 1	(5000, 0.05, 2)	500.0	500.0	OK
Cas nominal 2	(10000, 0.10, 1)	1000.0	1000.0	OK
Cas limite : Seuil	(5000, 0.05, 1)	250.0	250.0	OK
Cas limite : Zéro semaine	(5000, 0.05, 0)	0.0	0.0	OK
Cas limite : Semaines négatives (erreur métier)	(5000, 0.05, -2)	0.0 (ou erreur)	0.0	À re- voir*
Cas extrême : Grande valeur	(1000000, 0.5, 52)	26 000 000	26 000 000	OK

Analyse : Le cas « Semaines négatives » révèle une faille : l'algorithme ne protège pas contre une saisie aberrante (retour dans le temps). En maintenance, on ajoutera une vérification `if (semaines < 0) return -1; // Code erreur` ou une assertion. C'est ça, la valeur des tests : trouver les trous avant le client.

LESAVAIS-TU ? Le Cameroun numérique suit le cycle de vie

Au Cameroun, des startups à succès comme GiftedMom (suivi de grossesse par SMS en zone rurale) ou Agro-Hub (mise en relation agriculteurs/acheteurs) n'ont pas codé « à l'aveugle ». Elles ont passé des mois en analyse de besoins sur le terrain (parler aux sages-femmes, aux cultivateurs), ont conçu des algorithmes robustes (gestion hors-ligne, synchronisation différée), codé en Java/Kotlin/Swift, testé avec de vrais utilisateurs (tests d'acceptation) et assurent une maintenance continue (nouvelles versions Play Store régulières). Le cycle de vie n'est pas une théorie d'école : c'est la différence entre un prototype qui plante et un service qui sauve des vies ou nourrit des familles.

EXPÉRIENCE TP Guidé : Simuler le cycle de vie sur papier (20 min)

Objectif : S'approprier les 5 phases sans ordinateur. Matériel : Feuille A4, stylo, cerveau.
Consigne : En binôme, concevez l'algorithme (pseudo-code) pour « Déterminer le bénéficiaire du tour de la semaine » dans la tontine de Mme Ngo.

1. Analyse (3 min) : Listez les entrées (liste membres, index tour actuel), la sortie (nom du bénéficiaire), la règle (rotation circulaire, si membre sorti → passer au suivant).
2. Conception (7 min) : Écrivez l'algorithme en pseudo-code clair (Variables, Début, Pour/Si, Fin). Ne codez pas en C !
3. Codage mental (3 min) : L'un dicte l'algo, l'autre « exécute » sur un exemple papier (5 membres : A, B, C, D, E ; tour actuel = 2 → C). Vérifiez.
4. Test (5 min) : Testez les cas : Liste vide, 1 seul membre, membre bénéficiaire déjà sorti (ex : C a déjà eu son tour et est retiré de la liste active). Corrigez l'algo si bug.
5. Maintenance (2 min) : Comment modifier l'algo si on veut gérer une « priorité urgence » (un membre peut demander à passer devant moyennant majoration) ?

Restitution : Un binôme présente son algo au tableau, la classe joue le rôle de « testeur » en proposant des cas tordus.

2 DE L'ALGORITHME AU PROGRAMME : LANGAGES ET TRADUCTION

Dans la section précédente, nous avons vu qu'un logiciel naît d'un besoin, qu'on le conçoit (analyse, algorithmes) avant de le réaliser. Nous voici maintenant au cœur de cette réalisation : **comment transformer un algorithme écrit en français structuré (pseudo-code) en un programme que l'ordinateur comprend et exécute ?** C'est le passage de la logique humaine à l'exécution machine.

2.1 Du pseudo-code au langage de programmation : pourquoi changer de notation ?

Rappelons-nous : un **algorithme** est une suite finie et non ambiguë d'instructions pour résoudre un problème. En classe, nous l'écrivons en **pseudo-code** (ou Français Structuré) : SI ... ALORS ... SINON ... FIN SI, POUR i DE 1 À 10 FAIRE, AFFICHER. Cette notation est indépendante de toute machine : elle sert à réfléchir, à communiquer entre humains, à valider la logique avant de coder.

DÉFINITION Pseudo-code (Français Structuré)

Notation algorithmique normalisée, proche du langage naturel, utilisant des mots-clés réservés (DEBUT, FIN, SI, TANTQUE, POUR, AFFICHER, LIRE, <-->) pour décrire la logique d'un

programme sans se soucier de la syntaxe stricte d'un langage machine. Il est exécutable par l'esprit humain, pas par le processeur.

L'ordinateur, lui, ne comprend que le **binaire** (suites de 0 et 1). Écrire directement en binaire est impossible pour un humain (trop lent, trop d'erreurs, illisible). Nous avons donc inventé des **langages de programmation** : des notations formelles, strictes, textuelles, qui servent d'interface entre la pensée algorithmique et le matériel.

PROPRIÉTÉ Niveaux de langage

On classe les langages selon leur proximité avec le matériel (bas niveau) ou avec l'humain (haut niveau) :

- Langage machine (1^{re} génération) : Binaire pur (10110000 01100001). Exécuté directement par le CPU. Incompréhensible, non portable.
- Assembleur (2^{de} génération) : Mnémoniques textuels (MOV AL, 61h). Correspondance 1-pour-1 avec le langage machine. Toujours dépendant du processeur (x86, ARM). Utilisé pour pilotes, systèmes embarqués, optimisation extrême.
- Langages de haut niveau (3^{me} génération et +) : Proches de l'anglais et des maths (if (x > 5) { ... }, print("Grand")). Indépendants de la machine (portables). Un même code source peut tourner sur Windows, Linux, Mac, téléphone, après traduction adaptée. Exemples : **C**, **C++**, **Java**, **Python**, **JavaScript**, **C#**.

À RETENIR Choix pédagogique en 3^{me}

Au BEPC, nous manipulons principalement le pseudo-code (pour l'algorithmique pure) et nous découvrons la syntaxe concrète en C et en JavaScript (langages au programme officiel), avec Python souvent utilisé comme passerelle pédagogique pour l'interprétation. L'essentiel n'est pas de mémoriser la syntaxe par cœur, mais de comprendre la correspondance structurelle : un **SI** pseudo-code devient un **if** en C/JS/Python, une boucle **POUR** devient un **for** ou **while**, une variable s'écrit **x = 10** (Python/JS) ou **int x = 10;** (C).

2.2 La traduction : Compilation vs Interprétation

Un **code source** (fichier texte .c, .js, .py, .java) ne s'exécute pas tout seul. Il doit être traduit en instructions machine. Il existe deux grandes stratégies : la **compilation** et l'**interprétation**.

COMPILATION (C, C++, Java*)



INTERPRÉTATION (JavaScript, Python, Bash)



Schéma comparatif : Compilation (bloc unique, fichier exécutable intermédiaire) vs Interprétation (flux continu, pas de fichier exécutable persistant).

*Java compile vers du bytecode pour machine virtuelle (JVM), approche hybride.

DÉFINITION **Compilateur**

Programme (ex : `gcc` pour C/C++, `javac` pour Java) qui analyse la totalité du code source, vérifie la syntaxe, optimise, et produit un fichier exécutable (code machine natif ou bytecode) distinct du source. L'exécution a lieu après, séparément. Avantage : rapidité d'exécution, détection précoce des erreurs de syntaxe. Inconvénient : cycle Éditer → Compiler → Lancer plus long ; l'exécutable est souvent spécifique à un OS/architecture.

DÉFINITION **Interpréteur**

Programme (ex : `python3`, `node` pour JS, moteur JS du navigateur) qui lit le code source instruction par instruction, la traduit à la volée en actions machine, et l'exécute immédiatement. Pas de fichier exécutable intermédiaire persistant. Avantage : cycle Éditer → Lancer immédiat, portabilité totale (même source partout), idéal pour scripts, tests, apprentissage. Inconvénient : exécution plus lente (analyse répétée), erreurs de syntaxe découvertes seulement quand la ligne est atteinte.

EXEMPLE **Exemple chiffré : Même algorithme, trois syntaxes (Pseudo-code, C, JavaScript)**

Algorithme : « Lire un entier x. Si x est strictement supérieur à 5, afficher "Grand", sinon afficher "Petit ou égal". »

```

/* Fichier : test_if.c */
#include <stdio.h>           // Bibliothèque entrée/sortie

int main(void) {            // Point d'entrée obligatoire
    int x;                  // Déclaration + TYPE obligatoire
    printf("Entier x : ");
    scanf("%d", &x);        // Lecture formatée (adresse &x)
    if (x > 5) {             // Parenthèses OBLIGATOIRES, accolades pour bloc
        printf("Grand\n");
    } else {
        printf("Petit ou egal\n");
    }
    return 0;               // Code retour au système
}

```

Listing 3 – C (Compilé avec gcc)

```

// Fichier : test_if.js
const readline = require('readline').createInterface({
    input: process.stdin,
    output: process.stdout
});

readline.question("Entier x : ", (input) => {
    const x = parseInt(input); // Conversion chaîne -> entier
    if (x > 5) {               // Parenthèses OBLIGATOIRES, accolades recommandées
        console.log("Grand");
    } else {
        console.log("Petit ou egal");
    }
    readline.close();
});

```

Listing 4 – JavaScript (Interprété par navigateur / Node.js)

```

# Fichier : test_if.py
x = int(input("Entier x : ")) # Lecture clavier (str -> int)
if x > 5:                     // Deux-points OBLIGATOIRE, indentation OBLIGATOIRE
    print("Grand")           // Bloc SI (4 espaces)
else:
    print("Petit ou egal")    // Bloc SINON

```

Listing 5 – Python 3 (Interprété)

```

Algorithme ComparerX
Variable x : Entier
Debut
    Afficher "Entier x : "
    Lire x
    Si x > 5 Alors
        Afficher "Grand"
    Sinon
        Afficher "Petit ou egal"
    Fin Si
Fin

```

Listing 6 – Pseudo-code (Français Structuré)

-
-
-
-

2.3 Code source, code objet, exécutable : le vocabulaire précis

Terme	Description	Exemples d'extensions
Code source	Texte écrit par le programmeur, lisible par l'humain.	.c, .cpp, .js, .py, .java, .cs
Code objet	Résultat intermédiaire de la compilation (binaire non lié, symboles non résolus).	.o (Linux), .obj (Windows)
Édition de liens (Linking)	Fusion du code objet avec les bibliothèques (ex : <code>printf</code> , <code>math.h</code>) pour résoudre les adresses.	—
Exécutable	Fichier final prêt à être chargé en mémoire par l'OS et exécuté par le CPU.	a.out, .exe, .elf (sans extension souvent sous Linux)
Bytecode	Code intermédiaire portable (machine virtuelle).	.class (Java), .pyc (Python cache)

ATTENTION Ne pas confondre

- Fichier source ≠ Fichier exécutable. On ne modifie jamais l'exécutable directement (binaire). On modifie le source, puis on recompile/relance.
- En Python, les fichiers .pyc (dans `__pycache__`) sont du bytecode mis en cache pour accélérer le démarrage suivant. Ce sont des fichiers temporaires, on ne les édite pas.
- `gcc programme.c` fait les deux étapes : compilation → édition de liens → produit a.out (ou `programme.exe`).

2.4 Les trois familles d'erreurs : les identifier pour les corriger

Quand « ça ne marche pas », il faut savoir **où** et **quand** l'erreur survient. C'est la compétence n°1 du développeur débutant.

PROPRIÉTÉ Classification des erreurs

1. Erreurs de compilation (Syntaxiques) : Détectées par le compilateur (ou l'interpréteur avant d'exécuter la ligne fautive en Python/JS). Le programme ne démarre pas.
 - Oubli de ; (C, JS), : (Python), parenthèses/accolades déséquilibrées.
 - Variable non déclarée (C), mot-clé mal orthographié (`whille`, `pritrn`, `consloe.log`).
 - Indentation incorrecte (Python `IndentationError`).

Message typique : `error: expected ';' before '}' token (C)` ou `SyntaxError: invalid syntax (Python)` ou `Uncaught SyntaxError (JS)`.

2. Erreurs d'exécution (Runtime / Exceptions) : Le programme démarre, mais plante en cours de route sur une instruction impossible.
 - Division par zéro (`ZeroDivisionError`, `Floating point exception / Infinity` en JS).
 - Accès index hors borne tableau/liste (`IndexError`, `Segmentation fault / undefined` en JS).
 - Conversion invalide : `int("abc")` → `ValueError / NaN` en JS.
 - Fichier introuvable (`FileNotFoundError`).

Message typique : `Traceback (most recent call last): ... ZeroDivisionError: division by zero.`

3. Erreurs logiques (Sémantiques) : Le programme tourne sans planter, mais produit un mauvais résultat. Le code est syntaxiquement correct, l'algorithme est faux.
 - Condition inversée : `if (x < 5)` au lieu de `>`.
 - Boucle `while` qui ne s'arrête jamais (condition de sortie jamais vraie).
 - Variable mal initialisée (somme = 1 au lieu de 0).
 - Priorité d'opérateurs oubliée : `a + b * c` vs `(a + b) * c`.

Symptôme : « Mon programme affiche 15 au lieu de 20. »

MÉTHODE Débogage basique : la démarche en 3 étapes

1. Lire le message d'erreur (la trace). Ne pas paniquer. Repérer : Numéro de ligne, Type d'erreur (`SyntaxError`, `ZeroDivisionError`, `NameError`, `ReferenceError`), Description.

En C, la première erreur cascade souvent : corrige-la, recompile, les suivantes disparaissent souvent.

2. Vérifier la syntaxe locale (autour de la ligne). Deux-points : ? Parenthèses () {} équilibrées ? Indentation cohérente (Python) ? Point-virgule ; (C, JS) ? Guillemets fermés ? Noms de variables exacts (casse respectée : `sommeTotale` ≠ `sommetotale`) ?
3. Afficher les valeurs intermédiaires (`print` / `printf` / `console.log`). C'est la technique n°1 : insère des `print("DEBUG: x =", x)` avant la ligne qui plante ou donne un mauvais résultat. Suit le flux d'exécution. Retire-les une fois le bug corrigé.

EXPÉRIENCE TP guidé : Chasser les 3 types d'erreurs

Objectif : Provocer, observer, corriger une erreur de compilation, une erreur d'exécution, une erreur logique. Outils : Éditeur de code (VS Code, Geany, Thonny, ou replit.com en ligne) + Python 3 et Compilateur C (gcc via terminal ou IDE Code :: Blocks) et Navigateur / Node.js pour JS.

Étape 1 — Erreur de compilation (Python)

Copie ce code, sauvegarde `erreur_syntaxe.py`, lance-le.

```
x = 10
if x > 5      # Oublié les deux-points :
    print("Grand")
else
    print("Petit")
```

Observation : `SyntaxError: expected ':' sur la ligne 2`. **Correction :** Ajoute : après `if x > 5` et après `else`.

Étape 2 — Erreur d'exécution (C)

Fichier `div0.c` :

```
#include <stdio.h>
int main() {
    int a = 10, b = 0;
    int res = a / b; // Division par zéro !
    printf("Resultat : %d\n", res);
    return 0;
}
```

Compile : `gcc div0.c -o div0`. **Lance :** `./div0`. **Observation :** Erreur arithmétique (core dumped) ou `Floating point exception`. **Pas de message Python-style**, le programme s'arrête brutalement. **Correction :** Tester `if (b != 0)` avant de diviser.

Étape 3 — Erreur logique (JavaScript)

Fichier `moyenne_bug.js` : Calculer la moyenne de 3 notes.

```
let n1 = 12, n2 = 14, n3 = 16;
let moyenne = n1 + n2 + n3 / 3; // Piège priorité : division avant addition !
console.log("Moyenne :", moyenne); // Affiche 24.66... au lieu de 14
```

Observation : Programme tourne, affiche un nombre, mais faux. Débogage (Méthode « Débogage basique » étape 3) :

```
console.log("DEBUG : n1+n2+n3 =", n1+n2+n3); // 42
console.log("DEBUG : n3/3 =", n3/3); // 5.33
let moyenne = (n1 + n2 + n3) / 3; // Parentheses obligatoires
console.log("Moyenne corrigée :", moyenne); // 14
```

Interprétation : L'erreur logique est la plus sournoise : pas de message d'erreur, il faut tester avec des valeurs connues (jeu de test) et afficher les étapes.

2.5 Bonnes pratiques : écrire du code lisible et maintenable

Un programme se lit **beaucoup plus souvent qu'il ne s'écrit** (relecture, correction, travail en équipe, reprise dans 6 mois). Le style n'est pas cosmétique : c'est de la **prévention de bugs**.

À RETENIR Règles d'or du style (Clean Code niveau 3^{me})

1. Indentation rigoureuse : 4 espaces (Python, standard PEP 8 ; JS standard) ou 1 tabulation (C). Jamais mélange espaces/tabs. Un bloc SI/POUR/FONCTION doit être visuellement décalé.
2. Noms explicites (self-documenting code) : `somme_totale`, `note_max`, `calculer_moyenne()` au lieu de `x`, `n`, `f()`. Convention : snake_case (Python, C) : `ma_variable`, `ma_fonction()`. camelCase (Java, JS) : `maVariable`, `maFonction()`. Anglais recommandé pour les noms (standard international), français accepté au BEPC si cohérent.
3. Commentaires utiles : Expliquer le « pourquoi » (intention, contrainte métier, astuce), pas le « quoi » (le code le dit).

```
# MAUVAIS : x = x + 1 # On ajoute 1 à x
# BON : x = x + 1 // Passage au candidat suivant (tour de boucle)
```

4. Constantes nommées : `TAUX_TVA = 19.25` (majuscules + underscore) au lieu de `0.1925` « magique » au milieu du code.
5. Une instruction par ligne : Pas de `x=1; y=2; if x>y: print("ok")`.
6. Limite de ligne : ≤ 80–100 caractères (lisibilité sans défilement horizontal).

ATTENTION Le piège de l'indentation en Python vs C / JavaScript

En Python, l'indentation définit la structure du programme.

```
if x > 0 :
    print("Positif")
    print("Traitement suite") # Dans le SI
print("Fin")                # Hors du SI
```

Si tu décalages mal `print("Traitement suite")`, il sort du `if` → logique changée → bug silencieux (erreur logique).

En C et JavaScript, les accolades `{}` définissent les blocs. L'indentation est ignorée par le compilateur/interpréteur.

```
if (x > 0) {
    printf("Positif\n");
    printf("Traitement suite\n"); // Dans le SI grâce aux accolades
}
printf("Fin\n");                // Hors du SI
```

Conséquence : En C/JS, on peut écrire tout sur une ligne (compilé/exécuté), mais c'est illisible. En Python, on ne peut pas : l'indentation est la syntaxe. Adopte l'habitude universelle : toujours mettre les accolades `{}` même pour une seule instruction, et toujours indenter correctement.

EXERCICE RÉSOLU 2

De l'algorithme au code C complet : Calcul de factorielle

Énoncé : Écrire l'algorithme (pseudo-code) puis le programme C qui demande un entier $n \geq 0$ et affiche $n!$ (factorielle). Rappel : $0! = 1$, $n! = 1 \times 2 \times \dots \times n$. Gérer le cas $n < 0$ (erreur).

SOLUTION

1. Algorithme (Pseudo-code)

```
Algorithme Factorielle
Variables n, i, fact : Entier
Debut
    Afficher "Entier n (>= 0) : "
    Lire n
    Si n < 0 Alors
        Afficher "Erreur : n doit etre >= 0"
    Sinon
        fact <- 1
        Pour i de 1 a n Faire
            fact <- fact * i
        Fin Pour
        Afficher "Factorielle de ", n, " = ", fact
    Fin Si
Fin
```

Listing 7 – Algorithme Factorielle

2. Programme C (avec commentaires)

```
#include <stdio.h>
#include <stdlib.h> // Pour EXIT_SUCCESS / EXIT_FAILURE

int main(void){
    int n;          // Entier saisi
    long long fact = 1; // long long pour éviter débordement rapide (13! > 2^31)
    int i;

    printf("Entier n (>= 0) : ");
    if (scanf("%d", &n) != 1){ // Vérification lecture correcte
        fprintf(stderr, "Erreur de lecture.\n");
        return EXIT_FAILURE;
    }

    if (n < 0) {
        printf("Erreur : n doit etre >= 0\n");
        return EXIT_FAILURE; // Code retour non-nul = erreur
    }

    // Boucle factorielle : i de 1 à n inclus
    for (i = 1; i <= n; i++){
        fact = fact * i; // ou fact *= i;
    }

    printf("Factorielle de %d = %lld\n", n, fact);
    return EXIT_SUCCESS; // 0 = succès
}
```

Listing 8 – factorielle.c

3. Compilation et tests (Terminal Linux/Mac/WSL)

```
$ gcc -Wall -Wextra -std=c11 factorielle.c -o factorielle
$ ./factorielle
Entier n (>= 0) : 5
Factorielle de 5 = 120
$ ./factorielle
Entier n (>= 0) : 0
Factorielle de 0 = 1
$ ./factorielle
Entier n (>= 0) : -3
Erreur : n doit etre >= 0
$ ./factorielle
Entier n (>= 0) : 20
Factorielle de 20 = 2432902008176640000
```

Listing 9 – Session terminal

Analyse : Options `-Wall -Wextra` activent tous les avertissements (variables non utilisées, comparaisons signées/non-signées, etc.) : habitude pro à prendre dès maintenant. `-std=c11` fixe la norme C11. `long long` (entier 64 bits) permet d'aller jusqu'à 20!; au-delà, débordement (besoin bibliothèques grands entiers).

LESAVAIS-TU ? Histoire : Grace Hopper et le premier compilateur (1952)

Avant 1952, programmer signifiait câbler des fils ou écrire en code machine/octal. Grace Hopper (amirale US Navy, pionnière informatique) a développé le A-0 System, le premier compilateur : il traduisait des notations mathématiques symboliques en code machine pour l'UNIVAC I. On lui a dit : « Les ordinateurs ne font que de l'arithmétique, ils ne peuvent pas "programmer" ». Elle a répondu en créant FLOW-MATIC, premier langage utilisant des mots anglais (READ, WRITE, IF), ancêtre direct du COBOL (1959), encore utilisé aujourd'hui dans la banque mondiale. Elle a aussi popularisé le terme « bug » (papillon de nuit coincé dans un relais du Harvard Mark II, 1947). Morale : La compilation, c'est l'invention qui a permis à l'informatique de sortir des laboratoires pour devenir un outil universel.

EXERCICE 1 Entraînement : Traduction, Erreurs, Style

1. Traduction pseudo-code → JavaScript : Écris le programme JavaScript (Node.js) correspondant à cet algorithme :

```
Algorithme Signe
Variable x : Reel
Debut
  Afficher "Nombre : "
  Lire x
  Si x > 0 Alors Afficher "Positif"
  Sinon Si x < 0 Alors Afficher "Négatif"
  Sinon Afficher "Nul"
  Fin Si
Fin
```

2. Chasse aux erreurs (C) : Ce code C contient 4 anomalies (1 avertissement compilation / logique, 1 exécution, 1 logique, 1 style). Identifie-les et corrige-les.

```
#include <stdio.h>
int main() {
  int a = 10, b = 0;
  printf("Division : %d / %d = ", a, b);
  printf("%d\n", a / b); // Ligne 5
  if (a = 5) {           // Ligne 6
    printf("a vaut 5\n");
  }
  return 0;
}
```

3. Style : Réécris ce fragment Python en respectant les conventions (noms, indentation, commentaire utile, constante).

```
p=19.25;ht=150;ttc=ht*(1+p/100);print(ttc)
```

CORRIGÉ Exercice n

1. JavaScript (Node.js) :

```
const readline = require('readline').createInterface({
  input : process.stdin,
  output : process.stdout
});

readline.question("Nombre : ", (input) => {
  const x = parseFloat(input);
  if (x > 0) {
    console.log("Positif");
  } else if (x < 0) { // else if = Sinon Si
    console.log("Négatif");
  } else {
    console.log("Nul");
  }
  readline.close();
});
```

2. Anomalies C :

Ligne 5 – Exécution `a / b`: Division par zéro ($b = 0$). Le programme plante (Floating point exception).
Correction : Tester `if (b != 0)` avant de diviser, sinon afficher erreur.

Compilation/Logique `if (a = 5) :` = est une affectation (met 5 dans a, l'expression vaut 5 → vrai).
 gcc -Wall émet un avertissement suggest parentheses around assignment used as truth value. Il faut `==` (comparaison). **Correction** : `if (a == 5)`.

Logique Le programme affiche « a vaut 5 » car l'affectation `a=5` a changé la valeur de a (était 10).

Style `int main()` → préférer `int main(void)` pour prototypage strict. Ajouter `#include <stdlib.h>` si utilisation de `EXIT_SUCCESS/FAILURE`.

Code corrigé :

```
#include <stdio.h>
#include <stdlib.h>

int main(void){
  int a = 10, b = 2; // b != 0 pour test division
  if (b != 0) {
    printf("Division : %d / %d = %d\n", a, b, a / b);
  } else {
    printf("Erreur : division par zero.\n");
  }
  if (a == 5) { // Comparaison ==
    printf("a vaut 5\n");
  }
  return EXIT_SUCCESS;
}
```

3. Style Python (PEP 8) :

```
# Calcul TTC à partir du HT et taux TVA
TAUX_TVA = 19.25 # Constante : majuscules + underscore
prix_ht = 150.0 # Noms explicites, snake_case, type float explicite
prix_ttc = prix_ht * (1 + TAUX_TVA / 100)
print(f"Prix TTC : {prix_ttc :.2f}") # f-string formatée (2 décimales)
```

3 LES BRIQUES DE BASE : VARIABLES, TYPES ET STRUCTURES DE CONTRÔLE

3.1 Introduction : le vocabulaire de l'algorithme

Dans la section précédente, nous avons vu qu'un algorithme s'écrit dans un langage précis (pseudo-code) avant d'être traduit en langage machine. Mais quel que soit le langage, **tout programme repose sur les mêmes trois piliers** : stocker des données (variables), décider quoi faire selon des conditions (sélection), et répéter des actions (itération). C'est le « B-A-BA » de la programmation, valable du plus petit script au plus gros système d'exploitation.

Imagine que tu gères la caisse d'une tontine scolaire à Yaoundé. Tu dois : **mémoriser** le total des cotisations (variable), **tester** si un membre a payé (condition), **répéter** la lecture des 20 reçus (boucle). Sans ces trois briques, impossible d'automatiser la tâche. Maîtriser ces concepts, c'est acquérir le pouvoir de décomposer n'importe quel problème en instructions élémentaires que l'ordinateur saura exécuter.

3.2 Les variables : des boîtes étiquetées en mémoire

DÉFINITION Variable

Une **variable** est une zone de la mémoire vive (RAM) identifiée par un nom (identificateur), capable de contenir une seule valeur à la fois d'un type donné. Sa valeur peut changer au cours de l'exécution du programme (d'où le nom « variable »).

Contrairement aux mathématiques où $x = 5$ est une vérité immuable, en algorithmique $x \leftarrow 5$ signifie : « réserve une case mémoire nommée x et dépose-y la valeur 5 ». Plus tard, $x \leftarrow x + 1$ remplace l'ancienne valeur par la nouvelle.

PROPRIÉTÉ Règles de nommage et déclaration

- **Nom** : commence par une lettre (pas de chiffre), sans espace, sans accent (ex. `sommeTotale`, `nbEleves`, `_note`). Sensible à la casse en C/Python (`Note` $\neq$ `note`), mais généralement insensible en pseudo-code MINESEC (on évite toutefois les ambiguïtés).
- **Déclaration (pseudo-code)** : `Variable x : ENTIER` — réserve l'espace et impose le type.
- **Affectation** : `x <- 10` (flèche vers la variable). La lecture se fait de droite à gauche : on évalue l'expression à droite, puis on stocke dans la variable à gauche.

PROPRIÉTÉ Initialisation obligatoire

Toute variable doit recevoir une première valeur (initialisation) avant d'être utilisée dans un calcul. Sinon, elle contient une « valeur poubelle » (résidu mémoire) et le résultat devient imprévisible.

ATTENTION Affectation vs Égalité — Piège n°1

En algorithmique et en C : `=` est l'affectation (`x = 5` stocke 5), `==` est le test d'égalité (`if (x == 5)`). En pseudo-code MINESEC : `x <- 5` (affectation) et `x = 5` (test). Ne jamais confondre les deux : `x = x + 1` en C incrémente ; en maths c'est faux (`0 = 1`).

3.3 Les types de base : quel genre de valeur dans la boîte ?

Le type détermine : l'espace mémoire réservé, les valeurs autorisées, et les opérations possibles.

Type	Valeurs typiques	Opérations principales	Exemple déclaration
ENTIER	Nombres sans virgule : -12, 0, 42, 2025	+, -, *, / (division entière), MOD, DIV	Variable age : ENTIER
REEL	Nombres à virgule : 3.14, -0.5, 12.0	+, -, *, / (division réelle)	Variable moyenne : REEL
CARACTERE	Un seul symbole : 'A', '7', '@', 'é'	Concaténation, comparaison	Variable sexe : CARACTERE
CHAINE	Suite de caractères : "Yaoundé", "Classe 3ème"	Concaténation (+), longueur, extraction	Variable nom : CHAINE
BOOLEEN	VRAI ou FAUX	ET, OU, NON	Variable aPayé : BOOLEEN

EXEMPLE Choisir le bon type pour la tontine

- Nombre de membres (20) → ENTIER
- Montant d'une cotisation (1500 FCFA) → ENTIER (pas de centimes) ou REEL si précision décimale
- Nom du membre → CHAÎNE
- A-t-il payé ce mois-ci? → BOOLEEN
- Moyenne des cotisations → REEL (division peut donner décimal)

3.4 Structure SÉQUENCE : l'ordre naturel

Par défaut, les instructions s'exécutent **une après l'autre**, de la première à la dernière. C'est la structure la plus simple : pas de choix, pas de retour en arrière.

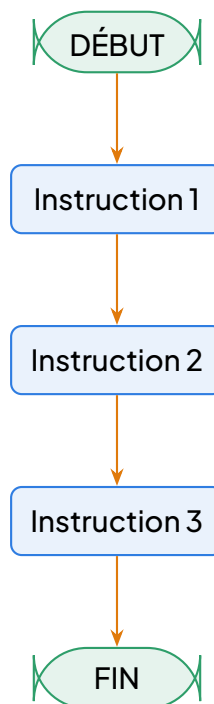


Diagramme de flux (flowchart) d'une structure SÉQUENCE : exécution linéaire.

```
// Calcul du périmètre d'un rectangle
Variable longueur, largeur, perimetre : ENTIER
DEBUT
    longueur <- 12    // Instruction 1
    largeur <- 7       // Instruction 2
    perimetre <- 2 * (longueur + largeur) // Instruction 3
    AFFICHER "Périmètre = ", perimetre // Instruction 4
```

FIN

Listing 10 – Exemple de séquence en pseudo-code

3.5 Structure SÉLECTION (Conditionnelle) : SI ... ALORS ... SINON

La sélection permet d'exécuter **un bloc d'instructions seulement si une condition est vraie**, et éventuellement un autre bloc si elle est fausse. C'est la prise de décision.

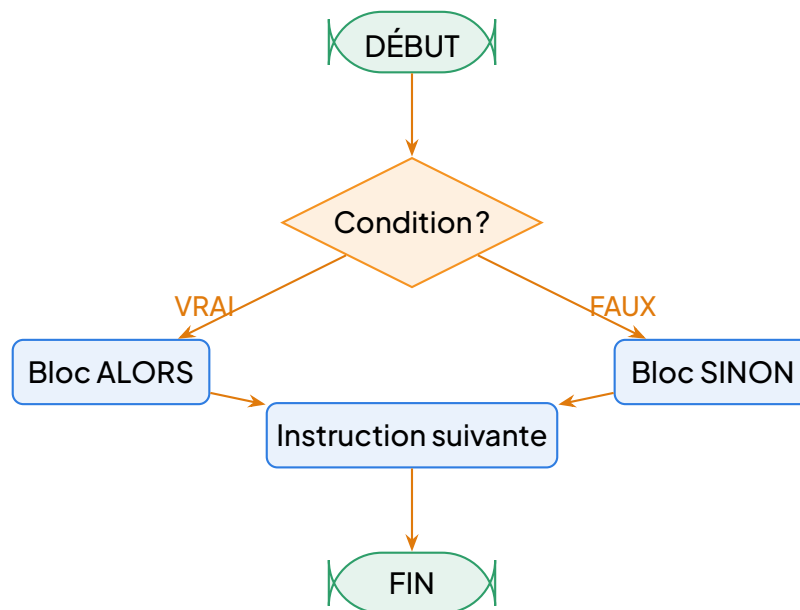


Diagramme de flux d'une structure SI ... ALORS ... SINON. Le losange = test; deux branches VRAI/FAUX.

PROPRIÉTÉ Syntaxe pseudo-code MINESEC

```
SI <condition> ALORS
  <instructions si VRAI>
SINON
  <instructions si FAUX>
FIN SI
```

La partie `SINON ...` est optionnelle (SI simple). Les blocs peuvent contenir d'autres SI (imbrication).

EXEMPLE Admission au BEPC — SI imbriqué

```
SI moyenne >= 10 ALORS
```

```
SI noteFrancais >= 10 ET noteMaths >= 10 ALORS
  AFFICHER "Admis au BEPC"
SINON
  AFFICHER "Ajourné (épreuve éliminatoire)"
FIN SI
SINON
  AFFICHER "Ajourné (moyenne < 10)"
FIN SI
```

ATTENTION Pièges classiques de la sélection

- Oublier `FIN SI` → erreur de structure.
- Confondre `=` (affectation) et `==` (test) dans la condition (en C); en pseudo-code, confondre `<-` et `=`.
- Écrire `SI x > 5 ET < 10` au lieu de `SI x > 5 ET x < 10`.
- Ordonner les tests du plus restrictif au plus large (sinon le premier « attrape » tout).

3.6 Structure ITÉRATION (Boucles) : répéter sans recopier

Une boucle exécute un bloc d'instructions **plusieurs fois**. Deux grandes familles selon qu'on connaît le nombre de tours à l'avance.

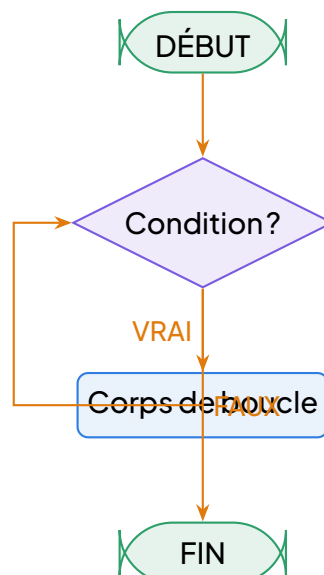


Diagramme de flux d'une boucle TANT QUE : test avant le corps. Flèche de retour (VRAI) vers le test.

MÉTHODE Choisir sa boucle : POUR vs TANT QUE

1. Je connais le nombre exact de tours à l'avance (ex. 20 membres, 12 mois, 100 lignes) → **POUR** (borne fixe).
2. Je m'arrête quand une condition devient fausse (ex. « tant que caisse < objectif », « tant que l'utilisateur ne tape pas 'Q' ») → **TANT QUE** (borne inconnue).
3. Attention : en TANT QUE, la condition doit éventuellement devenir fausse (variable modifiée dans le corps), sinon boucle infinie → plantage.

PROPRIÉTÉ Syntaxe pseudo-code — Boucle POUR

```
POUR i DE <début> A <fin> [PAS <pas>] FAIRE  
  <corps de boucle>  
FIN POUR
```

- i : variable de boucle (compteur), souvent ENTIER.
- PAS optionnel (par défaut +1). Peut être négatif pour décompter.

PROPRIÉTÉ Syntaxe pseudo-code — Boucle TANT QUE

```
TANT QUE <condition> FAIRE  
  <corps de boucle>  
FIN TANT QUE
```

Test avant chaque tour. Si condition fausse au départ → 0 tour.

EXEMPLE Calcul de la moyenne d'une tontine — 20 membres

```
Variable somme, montant, moyenne : REEL  
Variable i : ENTIER  
  
DEBUT  
  somme <- 0  
  POUR i DE 1 A 20 FAIRE  
    AFFICHER "Cotisation membre ", i, " : "  
    LIRE montant  
    somme <- somme + montant  
  FIN POUR
```

```
moyenne <- somme / 20
AFFICHER "Moyenne = ", moyenne
FIN
```

Trace (premiers tours) : $i = 1$, lu 1500 $\rightarrow$ somme=1500 ; $i = 2$, lu 2000 $\rightarrow$ somme=3500 ; ...
 $i = 20$, lu 1800 $\rightarrow$ somme=38500 ; moyenne = 1925.

EXEMPLE Saisie jusqu'à mot de passe correct — TANT QUE

```
Variable mdpSaisi : CHAÎNE
Variable ok : BOOLEEN

DEBUT
  ok <- FAUX
  TANT QUE ok = FAUX FAIRE
    AFFICHER "Mot de passe : "
    LIRE mdpSaisi
    SI mdpSaisi = "OnBuch2025" ALORS
      ok <- VRAI
    SINON
      AFFICHER "Incorrect, réessayez."
    FIN SI
  FIN TANT QUE
  AFFICHER "Accès autorisé."
FIN
```

Ici, le nombre de tours est inconnu à l'avance (dépend de l'utilisateur).

ATTENTION Boucle infinie — Danger n°1

```
Variable i : ENTIER
DEBUT
  i <- 1
  TANT QUE i <= 10 FAIRE
    AFFICHER i
    // OUBLI : i <- i + 1
  FIN TANT QUE
FIN
```

$\rightarrow$ Affiche « 1 » indéfiniment. Toujours modifier la variable de condition dans le corps (incrémenter compteur, lire nouvelle saisie, etc.).

3.7 Les opérateurs : calculer, comparer, combiner

Famille	Opérateurs	Signification	Exemple (VRAI/FAUX)
Arithmétiques	+ - * /	Addition, soustraction, multiplication, division	$7 / 2 = 3.5$ (REEL)
	DIV	Division entière (quotient)	$7 \text{ DIV } 2 = 3$
	MOD	Modulo (reste division entière)	$7 \text{ MOD } 2 = 1$
Relationnels	=	Égal à	$5 = 5 \rightarrow \text{VRAI}$
	<>	Différent de	$5 <> 3 \rightarrow \text{VRAI}$
	< > <= >=	Inférieur, supérieur, ≤, ≥	$10 >= 10 \rightarrow \text{VRAI}$
Logiques	ET	Conjonction : VRAI si les deux VRAI	$\text{VRAI ET FAUX} \rightarrow \text{FAUX}$
	OU	Disjonction : VRAI si au moins un VRAI	$\text{VRAI OU FAUX} \rightarrow \text{VRAI}$
	NON	Négation : inverse	$\text{NON VRAI} \rightarrow \text{FAUX}$

PROPRIÉTÉ Priorité des opérateurs (du plus fort au plus faible)

$() \rightarrow \text{NON} \rightarrow * / \text{ DIV MOD} \rightarrow + - \rightarrow < > <= >= = <> \rightarrow \text{ET} \rightarrow \text{OU}$

- Les parenthèses forcent l'évaluation (comme en maths).
- NON s'applique à l'opérande immédiat : $\text{NON } a \text{ ET } b = (\text{NON } a) \text{ ET } b$.
- En cas de doute : parenthéser pour la lisibilité.

LE SAVAIS-TU ? Le modulo (MOD) : le couteau suisse des programmeurs

L'opérateur MOD (modulo) donne le reste de la division entière. Applications courantes :

- Pair / Impair : $x \text{ MOD } 2 = 0 \iff \text{pair}$.
- Dernier chiffre : $x \text{ MOD } 10$ donne l'unité (ex. $2025 \text{ MOD } 10 = 5$).
- Cycles : jour de la semaine = $(\text{jourDepart} + \text{nbJours}) \text{ MOD } 7$.
- Conversion base : extraire bits (binaire), chiffres (hexadécimal).

C'est l'outil de base pour tout ce qui est « cyclique » ou « périodique ».

3.8 Exemple résolu complet : Gestion d'un mini-stock de cantine

EXERCICE RÉSOLU 3 Stock cantine — Variables, SI, POUR, TANT QUE

Énoncé. La cantine du lycée gère 5 produits. Pour chaque produit, on connaît le stock initial. Le programme doit : (1) afficher le stock initial de chaque produit ; (2) simuler 3 jours

de vente : chaque jour, on saisit le produit vendu (1 à 5) et la quantité ; on met à jour le stock si $\text{quantité} \leq \text{stock}$, sinon message « Rupture » ; (3) afficher le stock final.

Analyse.

- 5 produits → 5 variables distinctes `stock1..5` (ENTIER) (les tableaux ne sont pas au programme de 3ème).
- 3 jours fixes → boucle `POUR jour DE 1 A 3`.
- Choix produit + quantité → `LIRE` dans la boucle.
- Test stock suffisant → `SI qte <= stock ALORS ... SINON ...`

Algorithme (pseudo-code MINESEC).

```
ALGORITHME StockCantine
VARIABLES
    stock1, stock2, stock3, stock4, stock5 : ENTIER
    jour, choix, qte : ENTIER
DEBUT
    // 1. Initialisation (valeurs exemple)
    stock1 <- 50 ; stock2 <- 30 ; stock3 <- 40 ; stock4 <- 25 ; stock5 <- 60

    // Affichage initial
    AFFICHER "Stock initial :"
    AFFICHER "Produit 1 : ", stock1
    AFFICHER "Produit 2 : ", stock2
    AFFICHER "Produit 3 : ", stock3
    AFFICHER "Produit 4 : ", stock4
    AFFICHER "Produit 5 : ", stock5

    // 2. Simulation 3 jours
    POUR jour DE 1 A 3 FAIRE
        AFFICHER "--- Jour ", jour, " ---"
        AFFICHER "Produit vendu (1-5) : "; LIRE choix
        AFFICHER "Quantité : "; LIRE qte

        // Mise à jour selon choix
        SI choix = 1 ALORS
            SI qte <= stock1 ALORS stock1 <- stock1 - qte
            SINON AFFICHER "Rupture produit 1" FIN SI
        SINON SI choix = 2 ALORS
            SI qte <= stock2 ALORS stock2 <- stock2 - qte
            SINON AFFICHER "Rupture produit 2" FIN SI
        SINON SI choix = 3 ALORS
            SI qte <= stock3 ALORS stock3 <- stock3 - qte
            SINON AFFICHER "Rupture produit 3" FIN SI
        SINON SI choix = 4 ALORS
            SI qte <= stock4 ALORS stock4 <- stock4 - qte
            SINON AFFICHER "Rupture produit 4" FIN SI
        SINON SI choix = 5 ALORS
            SI qte <= stock5 ALORS stock5 <- stock5 - qte
            SINON AFFICHER "Rupture produit 5" FIN SI
        SINON
            AFFICHER "Produit inexistant"
        FIN SI
    FIN POUR
```

```
// 3. Stock final
AFFICHER "=== Stock final ==="
AFFICHER "P1:", stock1, " P2:", stock2, " P3:", stock3, " P4:", stock4, " P5:", stock5
FIN
```

Test manuel (trace partielle).

- Initial : [50, 30, 40, 25, 60]
- Jour 1 : choix=2, qte=10 → stock2=20 (OK)
- Jour 2 : choix=4, qte=30 → 30 > 25 → « Rupture produit 4 », stock4 inchangé
- Jour 3 : choix=1, qte=50 → stock1=0 (OK)
- Final : [0, 20, 40, 25, 60]

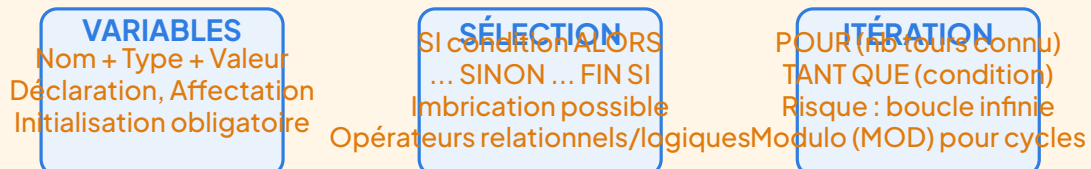
Remarque. Ce code répète la logique pour chaque produit. Dans la prochaine section (modularité), nous verrons comment factoriser avec une fonction pour éviter la duplication.

SOLUTION

Solution détaillée. Voir l'algorithme complet ci-dessus. Points clés validés :

- Déclaration explicite de toutes variables avec types.
- Initialisation avant utilisation (propriété respectée).
- Boucle POUR pour nombre de jours connu (3).
- SI / SINON SI / SINON pour dispatcher selon le produit.
- Test de garde `qte <= stock` avant déduction (évite stock négatif).
- Affichages clairs pour traçabilité.

3.9 Synthèse : les 3 piliers en une image mentale



À RETENIR Ce qu'il faut retenir pour le BEPC

1. Une variable = nom + type + valeur changeante. Toujours l'initialiser.
2. Quatre types de base : ENTIER, REEL, CHAÎNE/CARACTÈRE, BOOLEEN.
3. SÉQUENCE = défaut (linéaire). SÉLECTION = SI ... ALORS ... SINON ... FIN SI.

ITÉRATION = POUR (compteur) ou TANT QUE (condition).

4. Opérateurs : arithmétiques (+, -, *, /, DIV, MOD), relationnels (=, <>, <, >, <=, >=), logiques (ET, OU, NON).
5. Priorité : () → NON → * / DIV MOD → + - → relationnels → ET → OU.
6. MOD : reste division entière → pair/impair, cycles, dernier chiffre.
7. Pièges : <- vs =, boucle infinie, oubli FIN SI, condition mal parenthésée.

EXERCICE 2 Entraînement — Variables, Conditions, Boucles

1. Type et initialisation. Pour chaque donnée, donne le type le plus adapté et une initialisation correcte :
 - (a) Nombre d'élèves dans la classe.
 - (b) Prix unitaire d'un cahier (250 FCFA).
 - (c) Nom de l'élève.
 - (d) L'élève a-t-il rendu son livre? (Oui/Non).
 - (e) Moyenne de la classe (12,75).
2. Trace d'algorithme. Que affiche cet algorithme?

```
Variable x, y : ENTIER
DEBUT
  x <- 3; y <- 5
  SI x + y > 7 ALORS
    x <- x * 2
  SINON
    y <- y - 1
  FIN SI
  AFFICHER x, y
FIN
```

3. Boucle POUR. Écrire un algorithme qui calcule la factorielle $n! = 1 \times 2 \times \dots \times n$ pour un entier $n \geq 0$ saisi au clavier (rappel : $0! = 1$).
4. Boucle TANT QUE. Écrire un algorithme qui demande des notes (sur 20) jusqu'à ce que l'utilisateur saisisse une note négative, puis affiche la moyenne des notes valides (si au moins une note).
5. Modulo. Expliquer pourquoi $(\text{annee} \bmod 4 = 0 \text{ ET } \text{annee} \bmod 100 \neq 0) \text{ OU } (\text{annee} \bmod 400 = 0)$ détecte une année bissextile.

CORRIGÉ Exercice 3.1

1. (a) ENTIER, nbEleves <- 0 (b) ENTIER ou REEL, prix <- 250 (c) CHAINE, nom <- ""
(d) BOOLEEN, aRendu <- FAUX (e) REEL, moyenne <- 0.0
2. $x = 3, y = 5 \rightarrow x + y = 8 > 7$ VRAI $\rightarrow$ branche ALORS : $x \leftarrow 6$. Affichage : 6 5.
3. Factorielle (POUR) :

```

Variable n, i, fact : ENTIER
DEBUT
AFFICHER "n = "; LIRE n
fact <- 1
POUR i DE 1 A n FAIRE
    fact <- fact * i
FIN POUR
AFFICHER n, " ! = ", fact
FIN

```

4. Moyenne notes (TANT QUE) :

```

Variable note, somme : REEL
Variable nb : ENTIER
DEBUT
somme <- 0; nb <- 0
AFFICHER "Note (négative pour finir) : "; LIRE note
TANT QUE note >= 0 FAIRE
    somme <- somme + note
    nb <- nb + 1
    AFFICHER "Note suivante : "; LIRE note
FIN TANT QUE
SI nb > 0 ALORS AFFICHER "Moyenne = ", somme / nb
SINON AFFICHER "Aucune note saisie" FIN SI
FIN

```

5. Bissextile : Règle grégorienne. Divisible par 4 SAUF siècles non divisibles par 400. MOD 4 = 0 = multiple de 4. MOD 100 <> 0 = pas un siècle. MOD 400 = 0 = siècle multiple de 400 (ex. 2000 bissextile, 1900 non).

3.10 Transition vers la section 4

Nous savons maintenant stocker, décider, répéter. Mais un programme de 500 lignes tout dans le DEBUT...FIN devient illisible. La section 4 — **Modularité : Fonctions et Procédures** — nous apprendra à **découper** le problème en sous-problèmes indépendants (diviser pour régner), à réutiliser du code, et à rendre l'algorithme clair comme de l'eau de source. C'est le passage du « script » au « logiciel structuré ».

4 MODULARITÉ : FONCTIONS ET PROCÉDURES (DIVISER POUR RÉGNER)

Dans la section précédente, nous avons découvert les **briques de base** de tout algorithme : les variables, les types de données et les structures de contrôle (séquence, alternative, boucles). Avec ces outils, tu es déjà capable d'écrire des algorithmes complets. Mais imagine maintenant que tu doives écrire le programme de gestion complet d'une boutique à Yaoundé : saisie des produits, calcul des prix, gestion du stock, impression des reçus, statistiques de vente... Ton algorithme ferait des centaines de lignes ! Comment s'y retrouver ? Comment corriger une erreur sans tout casser ? C'est exactement le problème que résout la **modularité**.

4.1 Le problème : un algorithme unique devient illisible

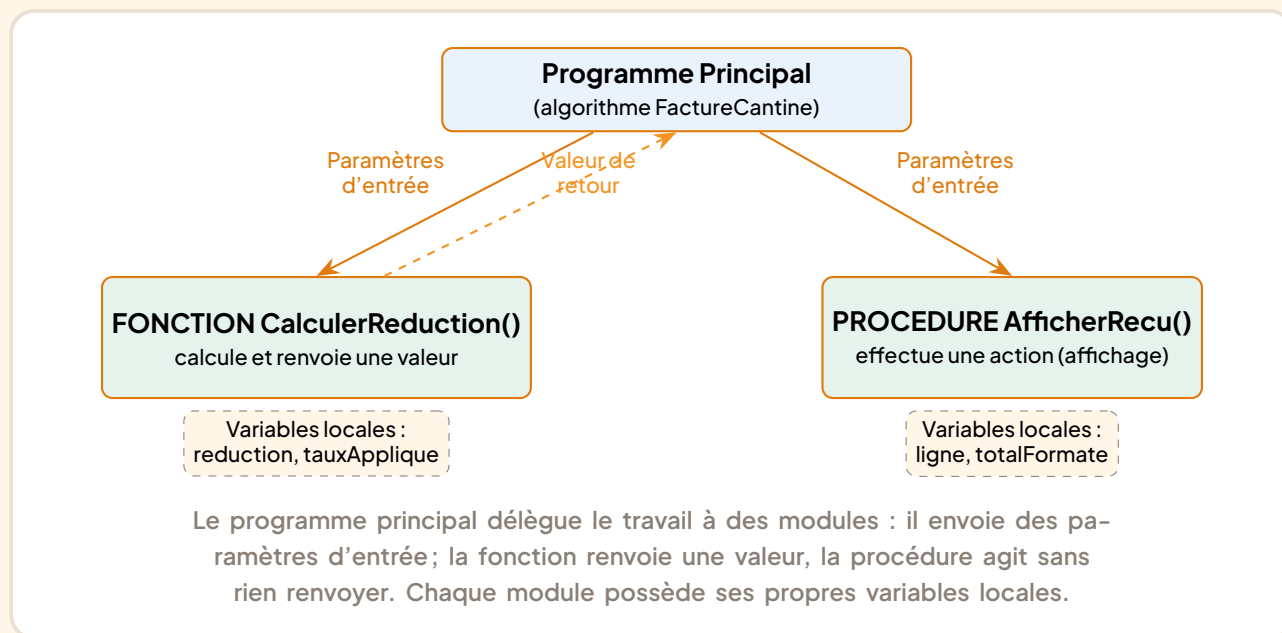
EXEMPLE Une situation de la vie courante

La cantine du collège veut un programme qui calcule la facture de chaque élève : prix du repas, réduction pour les boursiers, taxe, puis affichage du reçu. Si l'on écrit tout dans un seul bloc, on obtient un long algorithme où se mélangent calculs et affichages. Le jour où le taux de réduction change, il faut relire tout le programme pour trouver la bonne ligne... et corriger sans casser le reste !

Face à un problème complexe, l'être humain applique naturellement une stratégie très ancienne : diviser pour régner. Au lieu d'attaquer le gros problème d'un coup, on le découpe en petits sous-problèmes simples, que l'on résout un par un. En algorithmique, chaque sous-problème est confié à un **module**.

DÉFINITION Module (ou sous-algorithme)

Un **module** (ou **sous-algorithme**) est un morceau de programme nommé, qui réalise une tâche précise et qui peut être appelé (utilisé) par le programme principal ou par un autre module. Il existe deux sortes de modules : les **fonctions** et les **procédures**.



4.2 La fonction : un module qui calcule et renvoie une valeur

DÉFINITION Fonction

Une **fonction** est un module qui reçoit des **paramètres** (les entrées), effectue un traitement et renvoie obligatoirement une unique valeur résultat (le retour), grâce au mot-clé **RETOURNER**.

L'intuition est simple : une fonction est comme une machine à calculer. Tu lui donnes des ingrédients (les paramètres), elle te rend un plat (le résultat). Exemple : `CalculerInteret(capital, taux)` reçoit un capital et un taux, et renvoie l'intérêt calculé.

PROPRIÉTÉ Syntaxe générale d'une fonction en pseudo-code

```

FONCTION NomFonction(param1 : TYPE, param2 : TYPE) : TYPE_RETOUR
  VARIABLES
    variableLocale : TYPE
  DEBUT
    // traitements, calculs
    RETOURNER resultat
  FIN
  
```

Le type écrit après les deux-points finaux (ici `TYPE_RETOUR`) indique la nature de la valeur renvoyée (ENTIER, REEL, CHAÎNE, BOOLEEN...).

MÉTHODE Comment écrire une fonction, en 5 étapes

1. Nommer la fonction avec un verbe d'action : Calculer, Trouver, Convertir... Le nom doit annoncer clairement ce qu'elle fait.
2. Lister les paramètres entrants avec leurs types : de quoi la fonction a-t-elle besoin pour travailler ?
3. Déclarer les variables locales nécessaires aux calculs intermédiaires.
4. Écrire le corps de la fonction : les calculs, tests ou boucles qui produisent le résultat.
5. Renvoyer le résultat avec `RETOURNER`.

EXEMPLE Exemple chiffré : les frais d'un transfert d'argent

Une agence de transfert à Douala applique le tarif suivant : pour un montant inférieur ou égal à 5000 FCFA, les frais sont fixes : 150 FCFA ; au-delà, les frais valent 2 % du montant. Écrivons la fonction :

```
FONCTION FraisTransfert(montant : REEL) : REEL
  DEBUT
    SI montant <= 5000 ALORS
      RETOURNER 150
    SINON
      RETOURNER montant * 0.02
    FINSI
  FIN
```

Vérifions à la main : `FraisTransfert(3000)` renvoie 150 (car $3000 \leq 5000$). `FraisTransfert(20000)` renvoie $20000 \times 0,02 = 400$ FCFA. La fonction fait un seul travail : calculer les frais. Elle ne les affiche pas, elle les renvoie.

ATTENTION Principe de responsabilité unique

Une fonction ne doit faire qu'une seule chose, et la faire bien. Erreur classique du débutant : mélanger le calcul et l'affichage dans la même fonction. Si `FraisTransfert` affiche le résultat au lieu de le renvoyer, on ne peut plus réutiliser ce calcul ailleurs (par exemple pour additionner les frais de plusieurs transferts). La règle d'or : la fonction calcule et renvoie ; c'est le programme principal qui affiche.

4.3 La procédure : un module qui agit sans rien renvoyer

Parfois, on n'a pas besoin d'un résultat calculé : on veut simplement faire quelque chose — afficher un menu à l'écran, saisir les informations d'un membre, imprimer un reçu. C'est le rôle de

la procédure.

DÉFINITION Procédure

Une **procédure** est un module qui reçoit des paramètres, effectue un traitement (par exemple un affichage, une saisie, ou la modification d'une variable passée en paramètre de sortie) mais ne renvoie pas de valeur par `RETOURNER`.

EXEMPLE Deux procédures typiques

```
PROCEDURE AfficherMenu()
  DEBUT
    ECRIRE("1. Ajouter un membre")
    ECRIRE("2. Afficher les membres")
    ECRIRE("3. Quitter")
  FIN

PROCEDURE AfficherRecu(nom : CHAINE, total : REEL)
  DEBUT
    ECRIRE("---- RECU ----")
    ECRIRE("Client : ", nom)
    ECRIRE("Total à payer : ", total, " FCFA")
  FIN
```

On appelle ensuite ces procédures dans le programme principal simplement en écrivant leur nom : `AfficherMenu()` ou `AfficherRecu("Amina", 4500)`. Remarque la différence avec une fonction : l'appel d'une procédure est une instruction complète, alors que l'appel d'une fonction produit une valeur que l'on range dans une variable : `f <- FraisTransfert(20000)`.

4.4 Les paramètres : entrées et sorties

Les **paramètres** sont les données échangées entre le programme appelant et le module. On distingue :

- les **paramètres d'entrée** : une valeur fournie au module pour qu'il travaille (exemple : le `capital` et le `taux` de `CalculerInteret`);
- les **paramètres de sortie** : une variable que le module modifie pour communiquer un résultat (exemple : `nouveauSolde` mis à jour par une procédure `Deposer`).

PROPRIÉTÉ Passage par valeur et passage par référence (notion)

Lors de l'appel d'un module, les paramètres peuvent être transmis de deux façons :

- Passage par valeur : le module reçoit une copie de la donnée. S'il la modifie, la variable

d'origine du programme principal ne change pas.

- Passage par référence : le module reçoit l'adresse de la variable d'origine. Toute modification faite dans le module est répercutée dans le programme principal. C'est le mécanisme utilisé pour les paramètres de sortie.

Au niveau de la classe de 3ème, retiens simplement : une fonction communique son résultat par `RETOURNER` ; une procédure peut modifier des variables grâce aux paramètres de sortie.

4.5 La portée des variables : locales et globales

DÉFINITION Portée d'une variable

La **portée** d'une variable est la zone du programme où elle est visible et utilisable.

- Une variable **locale** est déclarée à l'intérieur d'un module : elle n'existe que pendant l'exécution de ce module et n'est visible nulle part ailleurs.
- Une variable **globale** est déclarée en dehors de tout module : elle est visible partout, par le programme principal et par tous les modules.

ATTENTION Pourquoi éviter les variables globales ?

Les variables globales semblent pratiques, mais elles sont dangereuses : n'importe quel module peut les modifier, et quand une erreur apparaît, il est très difficile de savoir qui a changé la valeur. Règle de bonne conduite : on privilégie les variables locales et les paramètres ; on n'utilise une variable globale qu'en dernier recours, pour une information vraiment partagée par tout le programme.

4.6 Les avantages de la modularité

À RETENIR Pourquoi découper un programme en modules ?

- Lisibilité : chaque module est court et porte un nom clair ; on comprend le programme en lisant les noms des appels.
- Réutilisabilité : une fonction écrite une fois peut être appelée autant de fois qu'on veut, dans le même programme ou dans un autre.
- Test unitaire : on peut tester chaque module indépendamment des autres, ce qui rend la chasse aux erreurs bien plus facile.

- Travail d'équipe : chaque élève du groupe peut écrire un module différent, puis on assemble le tout.
- Maintenance : si une règle change (par exemple le tarif des frais), on ne modifie qu'un seul module.

EXERCICE RÉSOLU 4 Gestion d'une caisse de tontine

Énoncé. Une tontine scolaire applique une cotisation fixe de 1000 FCFA par membre, plus une commission de 5 % sur le montant épargné. 1) Écris une fonction `Commission(montant : REEL) : REEL` qui renvoie la commission. 2) Écris une procédure `AfficherBilan(nom : CHAÎNE, montant : REEL)` qui affiche le nom du membre, la commission et le total à payer. 3) Écris le programme principal pour un membre « Ngo Bell » ayant épargné 20000 FCFA.

SOLUTION

Solution détaillée. 1) La fonction calcule et renvoie une seule valeur :

```
FONCTION Commission(montant : REEL) : REEL
DEBUT
    RETOURNER montant * 0.05
FIN
```

2) La procédure effectue des affichages, elle ne renvoie rien. Elle appelle elle-même la fonction `Commission` : c'est la réutilisabilité en action.

```
PROCEDURE AfficherBilan(nom : CHAÎNE, montant : REEL)
VARIABLES
    com, total : REEL // variables locales
DEBUT
    com <- Commission(montant)
    total <- 1000 + com
    ECRIRE("Membre : ", nom)
    ECRIRE("Commission : ", com, " FCFA")
    ECRIRE("Total à payer : ", total, " FCFA")
FIN
```

3) Le programme principal devient très court et très lisible :

```
ALGORITHME Tontine
DEBUT
    AfficherBilan("Ngo Bell", 20000)
FIN
```

Vérification : $\text{commission} = 20000 \times 0,05 = 1000 \text{ FCFA}$; $\text{total} = 1000 + 1000 = 2000 \text{ FCFA}$. Le programme affiche donc : Membre : Ngo Bell — Commission : 1000 FCFA — Total à payer : 2000 FCFA. Remarque que `com` et `total` sont des variables locales : elles n'existent que pendant l'exécution de la procédure.

EXPÉRIENCE TP guidé : construire et tester tes premiers modules

Objectif : découper un problème en modules et tester chaque module séparément. Matériel : papier et crayon, ou un logiciel d'algorithmique (ou Python si disponible en salle informatique).

1. Écris sur papier la fonction `FraisTransfert(montant : REEL) : REEL` vue plus haut.
2. Test unitaire : construis une table d'exécution avec trois montants : 2000, 5000 et 50000. Calcule à la main le résultat attendu pour chacun (150 ; 150 ; 1000).
3. Écris une procédure `AfficherFacture(montant : REEL)` qui appelle `FraisTransfert` puis affiche le montant, les frais et le total.
4. Écris le programme principal qui appelle `AfficherFacture(50000)`.
5. Observation : modifie le tarif (par exemple 3 % au lieu de 2 %). Que dois-tu changer ? Uniquement le corps de la fonction `FraisTransfert` : c'est tout l'intérêt de la modularité !

Interprétation : tester chaque module isolément permet de localiser immédiatement l'origine d'une erreur, au lieu de chercher dans tout le programme.

LESAVAIS-TU ? Ne réinvente pas la roue !

Les langages de programmation modernes sont livrés avec des **bibliothèques standard** : d'immenses collections de fonctions déjà écrites, testées et optimisées par des experts. En Python par exemple, la bibliothèque `math` fournit des fonctions comme `sqrt` (racine carrée), `random` permet de tirer des nombres au hasard, et `datetime` gère les dates et les heures. Un bon programmeur n'écrit que ce qui n'existe pas encore : pour tout le reste, il réutilise les modules existants. C'est la modularité poussée à l'échelle mondiale !

EXERCICE 3 À toi de jouer

- 1) Écris une fonction `Moyenne(n1 : REEL, n2 : REEL, n3 : REEL) : REEL` qui renvoie la moyenne de trois notes.
- 2) Écris une procédure `AfficherMention(moy : REEL)` qui affiche « Admis » si la moyenne est supérieure ou égale à 10, « Ajourné » sinon.
- 3) Écris le programme principal qui calcule puis affiche la mention d'un élève ayant obtenu 12, 8 et 14.
- 4) Indique, pour chaque variable que tu as utilisée, si elle est locale ou globale.

CORRIGÉ Exercice « À toi de jouer »

- 1) La fonction reçoit trois notes et renvoie leur moyenne :

```
FONCTION Moyenne(n1 : REEL, n2 : REEL, n3 : REEL) : REEL
DEBUT
    RETOURNER (n1 + n2 + n3) / 3
FIN
```

2) La procédure affiche la mention, sans rien renvoyer :

```
PROCEDURE AfficherMention(moy : REEL)
DEBUT
    SI moy >= 10 ALORS
        ECRIRE("Admis")
    SINON
        ECRIRE("Ajourne")
    FINSI
FIN
```

3) Le programme principal enchaîne les appels :

```
ALGORITHME Bulletin
VARIABLES
    m : REEL // variable du programme principal
DEBUT
    m ← Moyenne(12, 8, 14)
    ECRIRE("Moyenne : ", m)
    AfficherMention(m)
FIN
```

Vérification : $m = (12 + 8 + 14) / 3 = 34 / 3 \approx 11,33$, donc le programme affiche « Admis ». 4) La variable `m` est déclarée dans le programme principal : elle est locale au programme principal (elle n'est pas visible à l'intérieur des modules). Les paramètres `n1`, `n2`, `n3` et `moy` sont des variables locales à leurs modules respectifs. Aucune variable globale n'a été utilisée : c'est la bonne pratique.

5 VALIDER ET DOCUMENTER : TESTS, DÉBOGAGE ET QUALITÉ

Dans la section précédente, nous avons appris à diviser pour régner : un gros problème se découpe en petites fonctions et procédures, chacune facile à écrire et à comprendre. Mais une question essentielle reste en suspens : **comment être sûr que notre programme fait bien ce qu'on lui demande ?** Un programme qui s'exécute sans planter n'est pas forcément un programme correct. C'est toute la mission de cette dernière section : apprendre à **tester**, à **déboguer**, à **documenter** et à **sauvegarder** notre travail comme le font les vrais développeurs.

5.1 Pourquoi tester ? Le code sans test est du code cassé

DÉFINITION Test d'un programme

Tester un programme, c'est l'exécuter avec des données choisies à l'avance (appelées **jeu de test**) dont on connaît le résultat attendu, puis comparer le résultat obtenu au résultat attendu. Si les deux coïncident, le test passe (OK) ; sinon, il échoue (KO) et révèle un **bug** (une erreur dans le programme).

L'intuition est simple : imagine un menuisier de quartier à Douala qui fabrique une porte. Avant de la livrer, il vérifie qu'elle ferme, que la serrure tourne, qu'elle ne frotte pas le sol. S'il livre sans vérifier, le client découvrira les défauts à sa place — et ne le rappellera plus. En informatique, c'est pareil : **un code qui n'a jamais été testé doit être considéré comme cassé**, car on n'a aucune preuve qu'il fonctionne. Les erreurs ne se voient pas à l'œil nu dans le texte du programme : elles se révèlent à l'exécution.

DÉFINITION Régression

Une **régression** est l'apparition d'un bug dans une fonctionnalité qui marchait correctement avant une modification du programme. Par exemple, on corrige le calcul des frais et, sans s'en rendre compte, on casse l'affichage du solde. Rejouer les tests après chaque modification permet de détecter immédiatement les régressions.

5.2 Les types de tests : du petit morceau au logiciel complet

On ne teste pas tout de la même manière. On procède par couches, comme un mécanicien qui vérifie d'abord chaque pièce, puis le moteur assemblé, puis la voiture entière sur la route.

Type de test	Ce qu'on vérifie	Exemple (application de transfert)
Test unitaire	Chaque fonction ou procédure isolément	La fonction <code>FraisTransfert(montant)</code> renvoie-t-elle le bon montant de frais ?
Test d'intégration	Le bon fonctionnement des morceaux assemblés ensemble	La fonction <code>FraisTransfert</code> est-elle bien appelée par la procédure <code>EffectuerTransfert</code> , et le solde est-il bien diminué du montant + frais ?
Test système	Le logiciel complet répond-il au besoin de l'utilisateur ?	L'élève qui utilise l'application arrive-t-il à envoyer 5 000 FCFA à un camarade et à voir son reçu ?

À RETENIR Ordre logique des tests

On teste d'abord unitairement chaque brique, puis l'intégration des briques entre elles, puis le système complet. C'est l'avantage direct de la modularité vue à la section 4 : une fonction isolée est beaucoup plus facile à tester qu'un programme d'un seul bloc.

5.3 Construire un bon jeu de test : la stratégie des cas de test

La qualité d'un test ne dépend pas du nombre de cas, mais de leur choix. Un bon jeu de test explore trois familles de valeurs :

- les **valeurs nominales** : les cas normaux, typiques, « au milieu » (un montant ordinaire) ;
- les **valeurs limites** : les bornes exactes des conditions (0, la valeur maximale, la valeur juste en dessous et juste au-dessus d'un seuil) ;
- les **valeurs invalides** : ce que le programme ne devrait jamais recevoir (un montant négatif, du texte à la place d'un nombre) pour vérifier qu'il réagit proprement (message d'erreur, refus) au lieu de produire un résultat absurde.

MÉTHODE Construire un jeu de test complet

Pour tester correctement une fonction ou un programme :

1. Cas standard : choisir une valeur moyenne, représentative de l'usage normal.
2. Cas limites : tester chaque borne des conditions, incluse et exclue (par exemple pour la condition `montant >= 5000`, tester 4999, 5000 et 5001).
3. Cas d'erreur : fournir des entrées invalides (négatif, texte, vide) et vérifier la gestion prévue (message d'erreur clair).
4. Cas extrêmes : tester un très grand nombre, une liste vide, une boucle qui s'exécute zéro fois.

EXEMPLE Tester un calcul de remise — exemple chiffré

Une boutique applique la règle : SI `achat >= 10000` ALORS `remise <- 5%` de `achat` SINON `remise <- 0`. Voici le jeu de test minimal et les résultats attendus :

- `achat = 15000` (nominal) : remise attendue = $15000 \times 0,05 = 750$ FCFA ;
- `achat = 10000` (limite, seuil inclus) : remise attendue = 500 FCFA ;
- `achat = 9999` (limite, juste sous le seuil) : remise attendue = 0 FCFA ;
- `achat = -500` (invalid) : le programme doit signaler une erreur, pas calculer une re-

mise négative.

Ces quatre cas suffisent à piéger l'erreur classique consistant à écrire `>` au lieu de `>=` : le test avec 10000 échouerait et révélerait le bug.

ATTENTION Le piège du « chemin heureux »

Tester uniquement le cas où tout se passe bien (le « chemin heureux ») est insuffisant. Il faut aussi tester : les branches `SINON` des conditions, les boucles qui s'exécutent zéro fois (liste vide), les cas où une division par zéro pourrait se produire, et les entrées invalides. C'est presque toujours là que se cachent les bugs.

5.4 La table de test (table de traçabilité)

Pour organiser les tests et prouver qu'ils ont été faits, on utilise une **table de test** : chaque ligne est un cas de test, chaque colonne décrit une étape du raisonnement. Prenons la fonction `FraisTransfert` d'une application de transfert d'argent, dont le cahier des charges est :

- si `montant <= 0` : erreur, transfert refusé ;
- si `0 < montant < 5000` : frais = 100 FCFA ;
- si `5000 <= montant <= 10000` : frais = 250 FCFA ;
- si `montant > 10000` : frais = 500 FCFA.

Test N°	Montant	Condition évaluée	Frais attendus	Résultat prog.	Statut
1	3000 (nominal)	<code>0 < m < 5000</code>	100	100	OK
2	5000 (limite)	<code>5000 <= m <= 10000</code>	250	250	OK
3	10000 (limite)	<code>5000 <= m <= 10000</code>	250	500	KO
4	15000 (nominal)	<code>m > 10000</code>	500	500	OK
5	-100 (invalide)	<code>m <= 0</code>	Erreur	Erreur	OK

Table de test de la fonction `FraisTransfert`. Le test n°3 échoue (KO) : le programme renvoie 500 au lieu de 250 pour un montant de 10000, ce qui révèle un bug dans la borne de la deuxième condition (probablement `< 10000` écrit à la place de `<= 10000`).

Grâce à cette table, on voit immédiatement quel cas échoue, quelle condition est en cause, et on peut corriger précisément. C'est aussi un document à joindre à ton travail pour justifier ta démarche — une compétence exigée au BEPC comme en entreprise.

5.5 Le débogage : traquer le bug

Quand un test échoue, commence la phase de **débogage** (en anglais debugging) : retrouver et corriger l'erreur. Deux techniques de base, à la portée d'un élève de 3ème :

- la **trace d'exécution** : on suit à la main, ligne par ligne, la valeur de chaque variable (comme la table d'exécution vue en algorithmique) ;
- les **affichages intermédiaires** : on insère temporairement des instructions `Afficher` pour suivre l'évolution des variables pendant l'exécution, puis on les retire une fois le bug corrigé.

EXERCICE RÉSOLU 5 Déboguer la fonction `FraisTransfert`

Énoncé. Le test n°3 de la table ci-dessus a échoué : pour `montant = 10000`, le programme affiche 500 au lieu de 250. Voici le pseudo-code fautif. Trouve le bug en traçant l'exécution, puis corrige-le.

```
Fonction FraisTransfert(montant : reel) : reel
Debut
  Si montant <= 0 Alors
    Afficher("Erreur : montant invalide")
    Retourner -1
  FinSi
  Si montant < 5000 Alors
    Retourner 100
  SinonSi montant < 10000 Alors // <- zone suspecte
    Retourner 250
  Sinon
    Retourner 500
  FinSi
Fin
```

SOLUTION

Solution. Traçons l'exécution avec `montant = 10000` :

1. `10000 <= 0` ? Non.
2. `10000 < 5000` ? Non. On passe au `SinonSi`.
3. `10000 < 10000` ? Non (10000 n'est pas strictement inférieur à 10000). On tombe donc dans le `Sinon` : la fonction retourne 500. C'est bien le résultat fautif observé !

Le bug est identifié : la deuxième condition exclut la borne 10000, alors que le cahier des charges exige `5000 <= montant <= 10000`. Correction : remplacer `SinonSi montant < 10000 Alors` par `SinonSi montant <= 10000 Alors`. On rejoue ensuite toute la table de test : les 5 cas passent au statut OK. Leçon : le débogage compare toujours le comportement réel (tracé pas à pas) au comportement attendu (cahier des charges), et les bornes des conditions (`<` contre `<=`) sont les premières suspectes.

5.6 Documenter : écrire pour les humains, pas seulement pour la machine

Un programme est lu bien plus souvent qu'il n'est écrit — par toi-même dans trois mois, par un camarade, par un correcteur. La **documentation** rend le code compréhensible.

PROPRIÉTÉ Règles de documentation d'un programme

- Commentaire d'en-tête : en début de fichier ou de fonction, indiquer l'auteur, la date, le rôle du programme, ses entrées et ses sorties.
- Commentaires en ligne : ils expliquent le pourquoi d'une instruction délicate, pas le comment (le code dit déjà ce qu'il fait). Inutile d'écrire `// on ajoute 1 à i` !
- Nommage clair : des noms de variables explicites (`montantTransfert` plutôt que `x`), avec une convention constante : **CamelCase** (`FraisTransfert`, chaque mot commence par une majuscule) ou **snake_case** (`frais_transfert`, mots séparés par des tirets bas).

EXEMPLE Une fonction bien documentée

```
// -----  
// Fonction : FraisTransfert  
// Auteur   : A. Mbarga - Date : 12/03/2026  
// Role    : calcule les frais d'un transfert d'argent  
// Entree  : montant (reel), montant du transfert en FCFA  
// Sortie  : frais en FCFA, ou -1 si montant invalide  
// -----  
Fonction FraisTransfert(montant : reel) : reel  
Debut  
    Si montant <= 0 Alors  
        Retourner -1 // -1 signale une erreur à l'appelant  
    FinSi  
    Si montant < 5000 Alors  
        Retourner 100  
    SinonSi montant <= 10000 Alors  
        Retourner 250 // borne haute incluse : cahier des charges  
    Sinon  
        Retourner 500  
    FinSi  
Fin
```

5.7 Le versionning : garder l'historique avec Git (initiation)

Dernier réflexe de professionnel : ne jamais perdre son travail et pouvoir revenir en arrière. Plutôt que d'accumuler des fichiers `prog_final.txt`, `prog_final2.txt`, `prog_vraiment_final.txt`, on utilise un **gestionnaire de versions** comme **Git**. Il enregistre l'**historique** des modifications et permet de **collaborer** à plusieurs sur le même projet.

MÉTHODE Les trois commandes Git de base

1. `git init` : crée le dépôt (le « dossier surveillé ») du projet, une seule fois au début ;
2. `git add nom_fichier` : sélectionne les fichiers dont on veut enregistrer l'état actuel ;
3. `git commit -m "message"` : enregistre définitivement cet état dans l'historique, avec un message qui explique la modification (par exemple "correction des bornes de FraisTransfert").

Ainsi, chaque `commit` est une « photo » du projet : si une modification casse tout (régression!), on peut revenir à une photo précédente.

EXPÉRIENCE TP guidé : tester, déboguer, documenter une fonction

Objectif : appliquer toute la démarche qualité sur la fonction `FraisTransfert`. Matériel : un ordinateur avec un éditeur de texte (ou papier-crayon pour le pseudo-code).

1. Écriture : recopie la fonction documentée de l'exemple précédent, avec son en-tête complet (ton nom, la date du jour).
2. Préparation : reproduis la table de test à 5 lignes (3000, 5000, 10000, 15000, -100) et remplis la colonne « Frais attendus » avant d'exécuter quoi que ce soit.
3. Exécution : fais tourner le programme (ou trace-le à la main) pour chaque cas ; note le « Résultat programme » et le statut OK/KO.
4. Débogage : introduis volontairement l'erreur `< 10000` à la place de `<= 10000`. Quel test échoue ? Retrouve le bug en traçant l'exécution pas à pas, puis corrige.
5. Sauvegarde : si Git est disponible, fais `git init`, `git add`, puis un `git commit` avant la correction et un après, avec des messages clairs.

Observations attendues : le test avec le montant 10000 passe au statut KO avec la version fautive, puis redevient OK après correction. Interprétation : les valeurs limites sont les plus efficaces pour débusquer les erreurs de bornes.

LESAVAIS-TU ? Les tests, une compétence professionnelle clé

Dans les grandes entreprises du numérique, mais aussi dans les banques qui opèrent au Cameroun, on exige souvent qu'une grande partie du code (par exemple plus de 80 % des lignes) soit exécutée par au moins un test automatique avant toute mise en service du logiciel : c'est ce qu'on appelle la **couverture de tests**. Un logiciel bancaire mal testé pourrait envoyer de l'argent au mauvais client ! Savoir écrire un jeu de test, c'est déjà posséder une compétence recherchée sur le marché du travail.

À RETENIR L'essentiel de la section

- Un code non testé est un code suspect : on compare toujours résultat obtenu et résultat attendu.
- On teste par couches : unitaire (chaque fonction), intégration (l'assemblage), système (le besoin utilisateur).
- Un bon jeu de test mélange valeurs nominales, limites (bornes incluses et exclues) et invalides, sans oublier les cas extrêmes.
- La table de test trace chaque cas : entrée, condition, sortie attendue, résultat, statut OK/KO.
- Le débogage utilise la trace d'exécution ou des affichages intermédiaires pour comparer l'exécution réelle au cahier des charges.
- On documente (en-tête, commentaires utiles, nommage clair) et on versionne avec Git (`init`, `add`, `commit`) pour sécuriser son travail et collaborer.

EXERCICE 4 À toi de jouer

Une fonction `Mention(moyenne)` retourne « Ajourné » si $\text{moyenne} < 10$, « Passable » si $10 \leq \text{moyenne} < 12$, « Bien » si $12 \leq \text{moyenne} < 14$, et « Très Bien » si $\text{moyenne} \geq 14$. Construis une table de test complète (au moins 6 cas : nominaux, limites 10, 12, 14, et un cas invalide comme -3 ou 25), avec pour chaque ligne la condition évaluée, la sortie attendue et une colonne Statut à remplir.

CORRIGÉ Exercice : table de test de la fonction `Mention`

Cas proposés : moyenne = 11 (Passable, nominal); 10 (Passable, limite incluse); 9,99 (Ajourné, limite exclue); 12 (Bien, limite incluse); 14 (Très Bien, limite incluse); -3 (entrée invalide : le programme devrait signaler une erreur, car une moyenne est comprise entre 0 et 20); éventuellement 25 (invalide également). Chaque ligne doit indiquer la condition évaluée (par exemple $10 \leq m < 12$), la mention attendue, le résultat réel du programme et le statut OK/KO. Les bornes 10, 12 et 14 sont les cas les plus importants : elles détectent immédiatement toute confusion entre $<$ et $\leq$.

6 ACTIVITÉ D'INTÉGRATION

7 UTILISATION DES CONCEPTS DE BASE DU DÉVELOPPEMENT LOGICIEL

7.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de mobiliser l'ensemble des notions de base du développement logiciel étudiées dans cette leçon pour résoudre un problème concret de la vie courante. Plus précisément, tu devras :

- **Analyser** un énoncé de la vie réelle pour en extraire les **entrées**, les **sorties** et les **règles de calcul** (étape d'analyse du cycle de vie d'un logiciel) ;
- **Concevoir** un algorithme en pseudo-code utilisant des **variables** typées, des **structures conditionnelles** et une **fonction** avec paramètres et valeur de retour ;
- **Modulariser** la solution en séparant le calcul (fonction) de l'affichage (procédure), selon le principe « diviser pour régner » ;
- **Valider** le programme par un **jeu de tests** construit méthodiquement (cas normaux, cas limites, cas d'erreur) ;
- **Rédiger et justifier** ta démarche et tes choix, comme le ferait un vrai développeur qui documente son travail.

7.2 Situation

Simulateur de Tarif Moto-Taxi Yaoundé.

À Yaoundé, les motos-taxis (appelées « benskineurs ») sont un moyen de transport très utilisé, du quartier Nkolbisson jusqu'à Mokolo ou d'Ekounou au centre-ville. Mais les prix sont souvent discutés entre le client et le conducteur, ce qui crée des conflits quotidiens. Pour régler ce problème, la communauté urbaine a publié une **grille tarifaire officielle** et souhaite qu'un petit logiciel, installé sur un simple téléphone ou affiché dans les agences, calcule automatiquement le prix d'une course.

La grille officielle est la suivante :

- **Prise en charge** : 300 FCFA, qui incluent les 2 premiers kilomètres ;
- **Au-delà de 2 km** : chaque kilomètre supplémentaire coûte 100 FCFA ;
- **Majoration de nuit** : entre 22 heures (inclus) et 5 heures (exclu), le prix total est majoré de 50 % ;
- **Distance invalide** : si la distance saisie est négative, le logiciel doit signaler une erreur au lieu de calculer un prix absurde.

L'heure est saisie comme un **nombre entier** entre 0 et 23 (par exemple 14 pour 14 heures, 23 pour 23 heures). La distance est un **nombre réel** exprimé en kilomètres (par exemple 3,5 km).

Ton établissement a été choisi pour proposer un prototype. Tu es le développeur : à toi d'analyser le problème, de concevoir l'algorithme, de le modulariser, puis de le valider par des tests avant de le présenter à la classe.

7.3 Consignes

1. **Analyse.** Lis attentivement la situation. Identifie et rédige dans un tableau : (a) les **données d'entrée** du problème avec leur type (ENTIER ou REEL) ; (b) la **donnée de sortie** ; (c) les **constantes** du problème (valeurs fixées par la grille tarifaire) ; (d) les **règles de calcul** exprimées en français.
2. **Conception de la fonction.** Écris en pseudo-code la fonction `CalculerTarif(distance : REEL, heure : ENTIER) : REEL` qui retourne le prix de la course. Elle devra : traiter d'abord le cas d'erreur (distance négative), puis calculer le prix de base selon que la distance dépasse ou non 2 km, puis appliquer la majoration de nuit si nécessaire. Attention : la nuit correspond à `heure >= 22` ou `heure < 5`.
3. **Modularisation.** Écris la procédure `AfficherFacture(prix : REEL)` qui affiche proprement la facture du client (par exemple : « Prix de votre course : 800 FCFA »). Explique en deux phrases pourquoi il est préférable de séparer le calcul de l'affichage dans deux modules distincts.
4. **Validation.** Avant même d'exécuter quoi que ce soit sur machine, remplis la **table de tests** suivante en calculant à la main le résultat attendu pour chaque cas, puis en vérifiant que ton algorithme produit bien ce résultat (trace d'exécution) :

N°	Cas testé	distance	heure	Résultat attendu
1	Course courte de jour	1	14	...
2	Course moyenne de jour	5	10	...
3	Course courte de nuit	2	23	...
4	Longue course de nuit	10	2	...
5	Distance invalide	-3	12	...

5. **Justification.** Rédige un court paragraphe (5 à 8 lignes) qui justifie : (a) pourquoi la borne des 2 km est traitée par un test `distance <= 2` ; (b) pourquoi la nuit ne peut pas s'écrire avec un seul test du type `heure >= 22 ET heure < 5` ; (c) pourquoi il est indispensable de tester le cas de la distance négative.

Travail individuel ou en binôme, sur papier ou au tableau. La correction sera faite collectivement au vidéoprojecteur.

7.4 Ressources à mobiliser

À RETENIR Boîte à outils de la leçon

- Cycle de vie d'un logiciel : analyse → conception → codage → tests → mise en service. Ici, tu pratiques toutes les étapes sauf le codage en vrai langage.
- Variables et types : une variable possède un **nom**, un **type** (ENTIER, REEL, CHAÎNE, BOOLEEN) et une **valeur**. Les constantes de la grille (300, 100, 2, 22, 5) ne changent jamais.
- Structure conditionnelle :

```
Si condition Alors
    instructions
Sinon
    instructions
FinSi
```

- Fonction : module qui reçoit des **paramètres** et **retourne** un résultat : Fonction
Nom(param : TYPE) : TYPE.
- Procédure : module qui effectue une action (ici un affichage) sans retourner de valeur.
- Test de logiciel : on choisit des cas **normaux**, des cas **limites** (exactement 2 km, exactement 22 h) et des cas **d'erreur** (distance négative).
- Formule de la majoration : prix de nuit = prix de jour $\times (1 + \frac{50}{100})$ = prix de jour $\times 1,5$.

7.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 6 Simulateur de Tarif Moto-Taxi Yaoundé

Énoncé. Concevoir et valider un simulateur de tarif de moto-taxi : 300 FCFA les 2 premiers km, puis 100 FCFA par km supplémentaire, avec majoration de 50 % la nuit (22 h à 5 h) et rejet des distances négatives.

SOLUTION

Étape 1 : Analyse du problème.

On commence par le dictionnaire des données, comme dans toute analyse sérieuse :

Élément	Rôle	Type / Valeur
distance	Entrée	REEL, en km
heure	Entrée	ENTIER, entre 0 et 23
prix	Sortie	REEL, en FCFA
Prise en charge	Constante	300 FCFA (inclut 2 km)
Tarif kilométrique	Constante	100 FCFA/km au-delà de 2 km
Majoration nuit	Constante	+50 % si heure ≥ 22 ou heure < 5

Règles de calcul en français : si la distance est négative, c'est une erreur ; sinon, le prix de base vaut 300 FCFA si la distance est inférieure ou égale à 2 km, et $300 + 100 \times (\text{distance} - 2)$ au-delà ; enfin, si la course a lieu la nuit, on multiplie le total par 1,5.

Étape 2 : Conception de la fonction.

Fonction CalculerTarif(distance : REEL, heure : ENTIER) : REEL

Variable

prix : REEL

Debut

// Cas d'erreur : distance impossible

Si distance < 0 Alors

Retourner -1 // -1 est le code d'erreur choisi

FinSi

// Calcul du prix de base

Si distance ≤ 2 Alors

prix $\leftarrow 300$

Sinon

prix $\leftarrow 300 + 100 \times (\text{distance} - 2)$

FinSi

// Majoration de nuit : de 22h inclus à 5h exclu

Si (heure ≥ 22) Ou (heure < 5) Alors

prix $\leftarrow \text{prix} \times 1.5$

FinSi

Retourner prix

Fin

Étape 3 : Modularisation avec la procédure d'affichage.

Procédure AfficherFacture(prix : REEL)

Debut

Si prix = -1 Alors

Ecrire("Erreur : distance invalide, course refusée.")

Sinon

Ecrire("Prix de votre course : ", prix, " FCFA")

FinSi

Fin

Séparer le calcul (CalculerTarif) de l'affichage (AfficherFacture) permet de modifier la présentation de la facture sans toucher aux règles de tarification, et inversement : c'est le principe « diviser pour régner », qui rend le logiciel plus facile à tester et à faire évoluer.

Étape 4 : Validation par la table de tests.

N°	Cas testé	distance	heure	Résultat attendu
1	Course courte de jour	1	14	300 FCFA
2	Course moyenne de jour	5	10	600 FCFA
3	Course courte de nuit	2	23	450 FCFA
4	Longue course de nuit	10	2	1650 FCFA
5	Distance invalide	-3	12	erreur (-1)

Vérification par la trace d'exécution :

Cas 2 : $300 + 100 \times (5 - 2) = 300 + 300 = 600$ FCFA (jour, pas de majoration)

Cas 3 : $300 \times 1,5 = 450$ FCFA (2 km pile, donc tarif de base, mais 23 h = nuit)

Cas 4 : $(300 + 100 \times (10 - 2)) \times 1,5 = 1100 \times 1,5 = 1650$ FCFA

Les cinq résultats calculés à la main correspondent exactement à ce que produit l'algorithme : le jeu de tests est validé.

Étape 5 : Justification des choix.

(a) La borne des 2 km est traitée par `distance <= 2` car la grille dit que la prise en charge inclut les 2 premiers kilomètres : à exactement 2 km, on ne paie encore que 300 FCFA, et le supplément ne commence qu'au-delà. (b) La nuit ne peut pas s'écrire `heure >= 22 ET heure < 5`, car aucun entier n'est à la fois supérieur ou égal à 22 et strictement inférieur à 5 : cette condition serait toujours fausse et la majoration ne s'appliquerait jamais. La plage horaire « traverse minuit », il faut donc le `Ou`. (c) Le test de la distance négative est indispensable : sans lui, une erreur de saisie (par exemple -3 km) produirait un prix inférieur à 300 FCFA, voire négatif, ce qui serait absurde et ferait perdre de l'argent au conducteur. Tester les cas d'erreur fait partie du travail professionnel de validation.

ATTENTION Pièges fréquents rencontrés dans cette activité

- Écrire `distance < 2` au lieu de `distance <= 2` : le client de 2 km pile se verrait alors facturer 400 FCFA au lieu de 300 FCFA. Les **cas limites** sont les premiers à tester !
- Utiliser `Et` au lieu de `Ou` pour la plage de nuit (voir justification ci-dessus).
- Appliquer la majoration de 50 % en ajoutant seulement 50 FCFA : une majoration de 50 % est une multiplication par 1,5, pas une addition fixe.
- Oublier de traiter la distance négative, ou la traiter après le calcul : le test d'erreur doit venir en premier.

7.6 Bilan de l'activité

Cette activité t'a fait parcourir, à petite échelle mais de manière complète, le **cycle de vie d'un logiciel** : analyser un besoin réel (les conflits de prix entre clients et benskineurs), concevoir une solution algorithmique, la structurer en modules (une fonction et une procédure), puis la valider par un jeu de tests construit méthodiquement.

Elle a mis en évidence plusieurs idées essentielles du développement logiciel :

- Un bon programme commence par une **analyse sur papier** : identifier entrées, sorties et constantes avant d'écrire la moindre ligne évite la plupart des erreurs ;
- Les **structures conditionnelles** traduisent directement les règles de la vie réelle (bornes tarifaires, plages horaires), mais exigent une attention particulière aux **cas limites** (2 km exactement, 22 h exactement) ;
- La **modularité** (fonction de calcul + procédure d'affichage) rend le logiciel plus clair, plus facile à tester et plus facile à modifier si la grille tarifaire change l'année prochaine ;
- Un logiciel n'est jamais considéré comme terminé tant qu'il n'a pas été **validé par des tests**, y compris des tests de cas absurdes (distance négative) : c'est la démarche qualité des développeurs professionnels.

Tu peux désormais transposer cette démarche à d'autres situations camerounaises : calcul du prix d'un trajet en car interurbain, facturation d'une borne de recharge téléphonique, ou calcul d'une moyenne de notes avec mention. La méthode reste la même : analyser, concevoir, modulariser, tester, justifier.

8 MÉTHODES ET EXERCICES RÉSOLUS

8.1 Rappels essentiels : du besoin au programme

À RETENIR Le cycle de vie d'un logiciel

Un logiciel ne naît pas d'un coup : il suit un **cycle de vie** en étapes :

1. Analyse du besoin : comprendre ce que le logiciel doit faire ;
2. Conception : écrire l'**algorithme**, c'est-à-dire la suite ordonnée d'étapes qui résout le problème ;
3. Codage : traduire l'algorithme dans un **langage de programmation** (JavaScript, C, Python...) pour obtenir le **programme** ;
4. Tests et débogage : vérifier que le programme donne les bons résultats et corriger les erreurs (**bugs**) ;
5. Mise à jour et maintenance : faire évoluer le logiciel selon les nouveaux besoins.

L'ordinateur ne comprend que le **langage machine** (suites de 0 et de 1) : le programme écrit par le programmeur (le **code source**) doit donc être traduit par un logiciel spécial appelé **compilateur** ou **interpréteur** avant de pouvoir s'exécuter.

8.2 Méthode 1 : Appliquer les notions de développement logiciel à une situation concrète

MÉTHODE Résoudre un problème de la vie courante par le développement logiciel

Pour passer d'une situation concrète (boutique, cantine scolaire, club sportif) à un logiciel, suis toujours la même démarche :

1. Comprendre le besoin : formuler en une phrase ce que le logiciel doit faire (exemple : « calculer la moyenne d'un élève »).
2. Identifier les données : distinguer les **données d'entrée** (ce que l'utilisateur fournit), les **traitements** (calculs, comparaisons) et les **résultats** (ce que le logiciel affiche).
3. Choisir les variables : donner à chaque donnée un nom clair et un **type** adapté (entier, réel, chaîne de caractères).
4. Écrire l'algorithme : décrire les étapes en pseudo-code, dans l'ordre : Déclaration, Saisie, Traitement, Affichage.
5. Traduire en programme : choisir un langage (par exemple JavaScript) et transcrire l'algorithme en respectant la syntaxe.

6. Tester : exécuter le programme avec des valeurs connues dont on connaît le résultat à l'avance, puis corriger les erreurs (**débogage**).

Réflexe : un bon programme se lit dans l'ordre Saisie → Traitement → Affichage. Si l'une de ces trois étapes manque, la solution est incomplète.

EXERCICE RÉSOLU 7**La boutique de Mme Etoundi**

Mme Etoundi tient une boutique à Yaoundé. Elle vend des cahiers à 500 FCFA l'unité. Elle souhaite un petit programme qui demande le nombre de cahiers achetés par un client et affiche le montant total à payer.

1. Identifier les données d'entrée, le traitement et le résultat.
2. Écrire l'algorithme correspondant.
3. Traduire cet algorithme en JavaScript.

SOLUTION

1. Analyse du problème.

- Donnée d'entrée : le nombre de cahiers, que l'on nomme n (type : entier).
- Traitement : calcul du montant total : $\text{total} = n \times 500$.
- Résultat : affichage du montant total en FCFA.

On a besoin de deux variables : n (entier) et total (entier), ainsi que de la constante 500 (prix unitaire).

2. Algorithme.

```
Algorithme Boutique
Variables n, total : Entier
Début
    Écrire("Entrez le nombre de cahiers : ")
    Lire(n)
    total ← n * 500
    Écrire("Le montant à payer est : ", total, " FCFA")
Fin
```

L'ordre est respecté : saisie de n , puis traitement (calcul de total), puis affichage du résultat.

3. Traduction en JavaScript.

```
// Programme de calcul du montant à payer
var n = parseInt(prompt("Entrez le nombre de cahiers : "));
var total = n * 500; // prix unitaire : 500 FCFA
alert("Le montant à payer est : " + total + " FCFA");
```

Vérification : pour $n = 4$ cahiers, le programme calcule $4 \times 500 = 2000$ et affiche « Le montant à payer est : 2000 FCFA », ce qui est correct.

Conclusion : en suivant la démarche Saisie → Traitement → Affichage, on obtient un programme simple, correct et testé, qui répond exactement au besoin de Mme Etoundi.

8.3 Méthode 2 : Choisir correctement les variables et leurs types

MÉTHODE Déclarer les bonnes variables avec le bon type

1. Repérer chaque information manipulée par le problème (une information = une variable).
 2. Choisir un nom explicite : `moyenne`, `age`, `nomEleve` plutôt que `x`, `y`, `z`.
 3. Choisir le type selon la nature de la valeur :
 - **Entier** : un nombre sans virgule (âge, quantité, nombre d'élèves) ;
 - **Réel** : un nombre à virgule (moyenne, prix, taille) ;
 - **Chaîne de caractères** : un texte (nom, ville, mention) ;
 - **Booléen** : vrai ou faux (admis ou non).
 4. Initialiser la variable (lui donner une première valeur) avant de l'utiliser dans un calcul.
- À retenir : une moyenne sur 20 est un réel (elle peut valoir 12,75), jamais un entier ; un nom d'élève est une chaîne, jamais un nombre.

EXERCICE RÉSOLU 8 Les variables du bulletin de notes

Le proviseur d'un collège de Douala veut un programme qui gère les informations suivantes pour chaque élève : son nom complet, son âge, sa moyenne générale sur 20 et le fait qu'il soit admis ou non en classe supérieure.

1. Pour chaque information, proposer un nom de variable et son type, en justifiant.
2. Écrire la partie « Déclaration » de l'algorithme.

SOLUTION

1. Choix des variables.

Information	Nom de variable	Type	Justification
Nom complet	nomEleve	Chaîne	c'est un texte (lettres)
Âge	age	Entier	un âge est un nombre sans virgule
Moyenne sur 20	moyenne	Réel	elle peut avoir des décimales (ex. 13,50)
Admis ou non	admis	Booléen	seules deux valeurs possibles : vrai ou faux

2. Déclaration dans l'algorithme.

```

Algorithme Bulletin
Variables
    nomEleve : Chaîne
    age : Entier
    moyenne : Réel
    admis : Booléen
Début
    ...
Fin

```

Conclusion : chaque information possède un nom clair et un type adapté à sa nature ; c'est la première étape indispensable avant tout traitement.

8.4 Méthode 3 : Utiliser les structures de contrôle (alternative et répétitive)

MÉTHODE Choisir entre Si...Alors, Pour et Tant que

1. Un choix à faire ? Utilise la structure **alternative** : Si condition Alors ... Sinon ... FinSi. La condition doit être une comparaison claire (=, <, >, ≤, ≥, ≠).
2. Une action à répéter un nombre de fois connu à l'avance ? Utilise la boucle **Pour** : Pour i allant de 1 à n Faire ... FinPour.
3. Une action à répéter tant qu'une condition est vraie (nombre de tours inconnu) ? Utilise la boucle **Tant que** : Tant que condition Faire ... FinTantQue.
4. Vérifier la sortie de boucle : dans un Tant que, quelque chose dans le corps de la boucle doit modifier la condition, sinon le programme ne s'arrête jamais (boucle infinie).

Réflexe : pour calculer une somme ou un total, initialise toujours l'accumulateur à 0 avant la boucle, puis ajoute chaque valeur dans la boucle.

EXERCICE RÉSOLU 9 La cantine du collège

La cantinière d'un collège de Bafoussam veut calculer la recette du jour. Elle saisit le montant payé par chacun des 30 élèves inscrits, et le programme affiche la somme totale encaissée.

1. Quelle structure de contrôle faut-il utiliser ? Justifier.
2. Écrire l'algorithme complet.
3. Qu'affiche le programme si les trois premiers montants sont 200, 150 et 250 FCFA et que tous les autres élèves paient 100 FCFA ?

SOLUTION

1. Choix de la structure. Le nombre de répétitions est connu à l'avance : exactement 30 élèves. On utilise donc une boucle **Pour** (et non un **Tant que**, réservé aux cas où le nombre de tours est inconnu).

2. Algorithme.

```
Algorithme RecetteCantine
Variables i, montant, total : Entier
Début
    total ← 0
    Pour i allant de 1 à 30 Faire
        Écrire("Montant payé par l'élève ", i, " : ")
        Lire(montant)
        total ← total + montant
    FinPour
    Écrire("Recette totale : ", total, " FCFA")
Fin
```

Remarque l'initialisation `total ← 0` avant la boucle : sans elle, le résultat serait faux.

3. Calcul de la recette.

- Les trois premiers élèves paient : $200 + 150 + 250 = 600$ FCFA.
- Les $30 - 3 = 27$ autres élèves paient chacun 100 FCFA : $27 \times 100 = 2700$ FCFA.
- Total : $600 + 2700 = 3300$ FCFA.

Conclusion : le programme affiche « Recette totale : 3300 FCFA ». La boucle **Pour** est la structure adaptée car le nombre d'élèves (30) était connu avant le début du traitement.

8.5 Méthode 4 : Modulariser avec des fonctions et des procédures**MÉTHODE** Diviser un programme en sous-programmes

Principe « **diviser pour régner** » : un gros problème se découpe en petites tâches indépendantes.

1. Repérer les tâches répétées ou autonomes (un calcul, un affichage) : chacune devient un sous-programme.
2. Choisir entre fonction et procédure :
 - **Fonction** : elle calcule et retourne un résultat (mot-clé `Retourner` / `return`);
 - **Procédure** : elle effectue une action (affichage, saisie) sans retourner de valeur.
3. Définir les paramètres : ce sont les données dont le sous-programme a besoin pour travailler.
4. Appeler le sous-programme depuis le programme principal autant de fois que nécessaire.

Avantages à citer dans une copie : programme plus lisible, erreurs plus faciles à trouver, réutilisation du code, travail en équipe possible.

EXERCICE RÉSOLU 10 La moyenne de la classe

Un enseignant veut un programme qui calcule la moyenne de deux notes pour chaque élève de sa classe.

1. Écrire une fonction `moyenne` qui reçoit deux notes et retourne leur moyenne.
2. Écrire le programme principal qui demande les deux notes d'un élève, appelle la fonction et affiche le résultat.
3. Citer deux avantages de cette organisation.

SOLUTION

1. La fonction. Elle calcule un résultat : c'est donc bien une fonction (et non une procédure). Elle a deux paramètres réels et retourne un réel.

```
Fonction moyenne(a : Réel, b : Réel) : Réel
Début
    Retourner (a + b) / 2
Fin
```

2. Le programme principal.

```
Algorithme MoyenneClasse
Variables n1, n2, m : Réel
Début
    Écrire("Note 1 : ")
    Lire(n1)
    Écrire("Note 2 : ")
    Lire(n2)
    m ← moyenne(n1, n2)
    Écrire("La moyenne est : ", m)
Fin
```

8.6 Méthode 5 : Tester, déboguer et documenter un programme

MÉTHODE Valider un programme avant de le livrer

1. Préparer un jeu de tests : choisir des valeurs d'entrée dont on connaît le résultat exact à l'avance, en prévoyant des cas normaux et des cas limites (0, valeurs maximales, valeurs particulières).
2. Exécuter et comparer : si le résultat affiché diffère du résultat attendu, il y a un **bug**.
3. Déboguer : relire le code étape par étape (ou suivre les variables dans un tableau d'exécution) pour localiser l'instruction fautive, puis la corriger.
4. Re-tester après chaque correction : une correction peut créer une nouvelle erreur.
5. Documenter : écrire des **commentaires** dans le code (rôle de chaque variable, de chaque bloc) et une petite notice pour l'utilisateur.

Réflexe : un tableau d'exécution (une colonne par variable, une ligne par étape) est l'outil le plus sûr pour traquer une erreur de logique.

EXERCICE RÉSOLU 11 Le programme qui se trompe de mention

Voici un programme écrit par un élève pour attribuer une mention selon la moyenne :

```
Algorithme Mention
Variables m : Réel
Début
    Lire(m)
    Si m >= 10 Alors
        Écrire("Mention Passable")
    FinSi
    Si m >= 12 Alors
        Écrire("Mention Assez Bien")
    FinSi
Fin
```

1. Tester ce programme avec $m = 14$. Que constate-t-on ?
2. Expliquer l'erreur et proposer un programme corrigé.
3. Refaire le test avec $m = 14$ et $m = 8$ pour valider la correction.

SOLUTION

1. Test avec $m = 14$. La première condition $14 \geq 10$ est vraie : le programme affiche « Mention Passable ». Puis la deuxième condition $14 \geq 12$ est aussi vraie : il affiche « Mention Assez Bien ». Le programme affiche donc deux mentions contradictoires : c'est un bug.

2. Explication et correction. L'élève a utilisé deux tests indépendants au lieu d'une alternative avec `Sinon`. Il faut tester d'abord la condition la plus forte, puis utiliser `Sinon` pour les autres cas :

```
Algorithme MentionCorrige
Variables m : Réel
Début
  Lire(m)
  Si m >= 12 Alors
    Écrire("Mention Assez Bien")
  Sinon
    Si m >= 10 Alors
      Écrire("Mention Passable")
    Sinon
      Écrire("Moyenne insuffisante")
  FinSi
FinSi
Fin
```

3. Nouveaux tests.

- Avec $m = 14$: $14 \geq 12$ est vrai → affichage unique « Mention Assez Bien ». Correct.
- Avec $m = 8$: $8 \geq 12$ est faux, $8 \geq 10$ est faux → affichage « Moyenne insuffisante ». Correct.

Conclusion : le test avec une valeur connue a révélé le bug ; le débogage a montré que des conditions qui se recouvrent exigent une structure `Si ... Sinon ... FinSi` ; la re-validation confirme que le programme corrigé donne un seul résultat juste dans tous les cas.

8.7 Méthode 6 : Rédiger et justifier une démarche ou une conclusion**MÉTHODE****Rédiger une réponse complète et justifiée au BEPC**

1. Annoncer ce que l'on va faire : « Pour résoudre ce problème, j'utilise une boucle Pour car... ».

2. Dérouler la démarche dans l'ordre : analyse des données, choix des variables et des types, algorithme, traduction, test.
3. Justifier chaque choix par une règle du cours : type choisi selon la nature de la donnée, structure choisie selon que le nombre de répétitions est connu ou non, fonction choisie parce qu'un résultat est retourné.
4. Valider par un test chiffré : donner un exemple d'exécution avec le calcul complet.
5. Conclure par une phrase qui répond exactement à la question posée : « Le programme affiche donc... ».

Réflexe : une réponse sans justification ni exemple de test perd des points, même si l'algorithme est correct.

EXERCICE RÉSOLU 12 Accès à la salle informatique

Le club informatique d'un collège de Bafoussam réserve sa salle aux élèves de 12 ans et plus. Rédige une réponse complète et justifiée à la question : « Écrire un programme qui demande l'âge d'un élève et affiche s'il peut entrer dans la salle. »

SOLUTION

Démarche. Le programme doit prendre une décision selon une condition : il faut donc une structure alternative Si ... Sinon ... FinSi.

Choix des variables. Une seule donnée d'entrée : l'âge, noté `age`, de type **Entier** car un âge est un nombre sans virgule.

Algorithme.

```
Algorithme AccesSalle
Variables age : Entier
Début
    Écrire("Quel est votre âge ? ")
    Lire(age)
    Si age >= 12 Alors
        Écrire("Accès autorisé")
    Sinon
        Écrire("Accès refusé")
    FinSi
Fin
```

Justification. La condition « 12 ans et plus » se traduit par $\text{age} \geq 12$ (et non $\text{age} > 12$, qui exclurait les élèves de 12 ans exactement). Le `Sinon` garantit qu'un message s'affiche dans tous les cas.

Validation par des tests.

- Avec $\text{age} = 14$: $14 \geq 12$ est vrai → « Accès autorisé ». Correct.
- Avec $\text{age} = 12$: $12 \geq 12$ est vrai → « Accès autorisé ». Correct (cas limite).

- Avec `age = 10` : `10 ≥ 12` est faux → « Accès refusé ». Correct.

Conclusion : le programme ci-dessus répond au besoin du club : il autorise l'accès à tout élève de 12 ans et plus, et le refuse aux autres, comme le prouvent les trois tests effectués, y compris le cas limite de 12 ans.

LESAVAIS-TU ? D'où vient le mot « bug » ?

En 1947, les ingénieurs de l'ordinateur Harvard Mark II découvrirent qu'une panne était causée par un véritable insecte (une mite) coincé dans un relais de la machine. Depuis, le mot anglais bug (« insecte ») désigne toute erreur dans un programme, et déboguer signifie « enlever les insectes », c'est-à-dire corriger les erreurs. Aujourd'hui, au Cameroun comme partout dans le monde, les développeurs passent une grande partie de leur temps à tester et déboguer leurs applications avant de les mettre en service.

ATTENTION Les 5 erreurs qui coûtent des points au BEPC

1. Oublier d'initialiser une variable (par exemple `total <- 0` avant une boucle de somme) : le calcul devient faux ou imprévisible. Toute variable utilisée dans un calcul doit avoir reçu une valeur avant.
2. Confondre `>` et `≥` dans une condition : « 12 ans et plus » s'écrit `age ≥ 12`. Tester systématiquement le cas limite (ici 12) pour vérifier le bon sens de l'inégalité.
3. Choisir un mauvais type de variable : déclarer une moyenne en Entier fait perdre les décimales (12,75 devient 12) ; déclarer un nom en nombre est absurde. Justifie toujours le type par la nature de la donnée.
4. Écrire des tests indépendants qui se recouvrent au lieu d'un `Si ... Sinon ... FinSi` : le programme affiche alors plusieurs messages contradictoires (voir l'exercice sur les mentions).
5. Rendre un algorithme sans test ni conclusion : une copie doit contenir la démarche (analyse, variables, algorithme), un exemple d'exécution chiffré qui valide le résultat, et une phrase de conclusion qui répond à la question posée.

9

EXERCICES

9.1 Je vérifie mes connaissances

EXERCICE 5 QCM : Cycle de vie et vocabulaire

Parmi les propositions suivantes, une seule est exacte pour chaque question. Recopie le numéro de la question et la lettre de la bonne réponse.

1. Quelle phase du cycle de vie en V précède immédiatement la phase de codage (programmation)?
 - A. Recueil des besoins
 - B. Conception détaillée (algorithmes, structures de données)
 - C. Tests d'intégration
 - D. Maintenance corrective
2. Dans un algorithme, l'instruction `Afficher` correspond à quelle action concrète lors de l'exécution du programme?
 - A. Stocker une valeur en mémoire vive (RAM)
 - B. Envoyer un résultat vers la sortie standard (l'écran)
 - C. Lire une valeur saisie au clavier
 - D. Effectuer un calcul arithmétique
3. La structure de contrôle qui permet de répéter un bloc d'instructions tant qu'une condition reste vraie s'appelle :
 - A. `SI ... ALORS ... SINON ... FIN SI`
 - B. `POUR i DE 1 A N FAIRE ... FIN POUR`
 - C. `TANT QUE condition FAIRE ... FIN TANT QUE`
 - D. `SELON ... FAIRE ... FIN SELON`
4. Une fonction se distingue d'une procédure principalement parce qu'elle :
 - A. Ne reçoit aucun paramètre en entrée
 - B. Ne peut pas modifier de variables globales
 - C. Renvoie une valeur (un résultat) utilisable dans une expression
 - D. S'exécute plus rapidement

EXERCICE 6 Vrai ou Faux ? Justifie

Pour chaque affirmation, réponds Vrai ou Faux et justifie ta réponse en une phrase précise, en utilisant le vocabulaire du cours.

1. « L'algorithme est le code source écrit en langage C ou Python. »
2. « Une variable de type `Entier` peut stocker la valeur 12,5. »
3. « La phase de tests unitaires consiste à vérifier que le logiciel complet répond au cahier des charges du client. »
4. « Dans une boucle `POUR i DE 1 A 10 FAIRE`, la variable `i` prend successivement les valeurs 1, 2, ..., 10. »
5. « Le débogage consiste à écrire la documentation utilisateur du logiciel. »

EXERCICE 7 Définitions et correspondance

Associe chaque concept de la colonne de gauche à sa définition exacte dans la colonne de droite (écris les paires sous la forme : 1-c, 2-a, etc.).

Concept	Définition
1. Variable	a. Phase qui transforme le code source en programme exécutable
2. Compilation	b. Suite ordonnée et finie d'opérations permettant de résoudre un problème
3. Algorithme	c. Zone de mémoire nommée, d'un type défini, dont la valeur peut changer
4. Test unitaire	d. Vérification isolée du bon fonctionnement d'une fonction ou d'une procédure
5. Cahier des charges	e. Document qui décrit précisément ce que le logiciel doit faire

EXERCICE 8 Lecture d'algorithme : calcul d'une somme

Soit l'algorithme suivant écrit en pseudo-code :

```
Algorithme SommePremiersEntiers
Variable
    n, i, somme : Entier
Debut
    Afficher("Entrez un entier positif N : ")
    Saisir(n)
```

```

somme ← 0
Pour i de 1 à n faire
    somme ← somme + i
Fin Pour
Afficher("La somme des ", n, " premiers entiers est : ", somme)
Fin

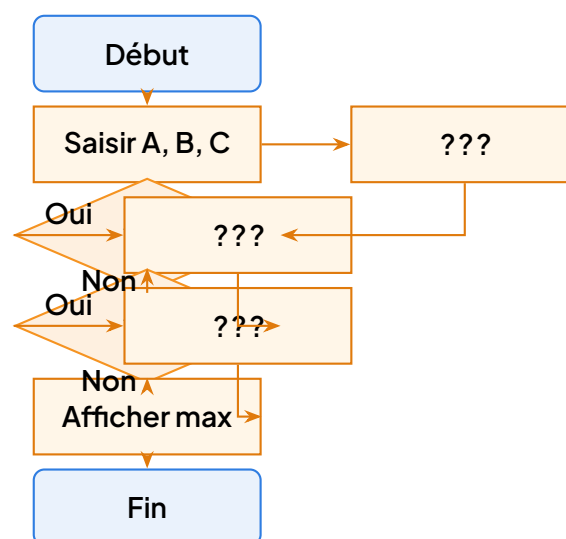
```

1. Quelle est la valeur finale de `somme` si l'utilisateur saisit 5 ? Détaille le calcul.
2. Quelle structure de contrôle est utilisée pour la répétition ?
3. Quel serait l'affichage si l'utilisateur saisit 0 ? (On admet que dans ce cas la boucle `POUR` n'est pas exécutée.)

9.2 Je m'entraîne

EXERCICE 9 Compléter un organigramme : maximum de trois nombres

L'organigramme ci-dessous détermine le maximum de trois nombres A , B , C . Les losanges (tests) et les rectangles (actions) sont vides : à toi de les compléter.



Organigramme à compléter : recherche du maximum de trois nombres

Recopie l'organigramme sur ta copie et complète les zones vides (tests et actions) pour que l'algorithme soit correct. Indication : on utilise une variable `max` initialisée avec la valeur de A , puis on la compare successivement à B puis à C .

EXERCICE 10 Écriture de fonction : année bissextile

Écris en pseudo-code (syntaxe du cours : Fonction, Debut, Fin, Retourner) une fonction `EstBissextile(annee : Entier) : Booleen` qui retourne VRAI si l'année passée en paramètre est bissextile, FAUX sinon.

Rappel de la règle du calendrier grégorien :

- une année est bissextile si elle est divisible par 4 et non divisible par 100 ;
- sauf si elle est divisible par 400 (alors elle est bissextile).

Indication : le reste de la division euclidienne s'écrit avec l'opérateur `MOD` ; par exemple, « `annee` est divisible par 4 » s'écrit `annee MOD 4 = 0`.

EXERCICE 11 Conception de jeux de tests : décision d'admission

On considère l'algorithme suivant qui détermine la décision du conseil de classe selon la moyenne d'un élève (sur 20) :

```
Algorithme Mention
Variable
    note : Reel
    mention : Chaîne
Debut
    Saisir(note)
    Si note >= 10 Alors
        mention <- "Admis"
    Sinon
        mention <- "Recalc"
    Fin Si
    Afficher(mention)
Fin
```

Propose un jeu de 4 tests pertinents sous forme de tableau. Pour chaque test, indique : le numéro du test, la valeur d'entrée (`note`), le résultat attendu (`mention`) et le type de test (cas normal, valeur limite basse, valeur limite haute, valeur invalide). Justifie brièvement le choix de chaque valeur.

EXERCICE 12 Algorithme complet : moyenne de la classe

Écris un algorithme complet (avec `Algorithme`, `Variable`, `Debut`, `Fin`) qui :

1. demande à l'utilisateur le nombre d'élèves `N` (entier strictement positif) ;
2. pour chaque élève de 1 à `N` (boucle `POUR`), demande sa note (réel compris entre 0 et 20) et l'ajoute à une variable `somme` ;
3. calcule la moyenne : `moyenne <- somme / N` ;

4. affiche la moyenne de la classe avec un message clair.

Veille à initialiser correctement la variable `somme` avant la boucle.

9.3 Je me prépare au BEPC

EXERCICE 13 Sujet type BEPC : gestion d'un stock de boutique — 12 pts

M. Kamga gère une petite boutique à Yaoundé. Il souhaite un programme simple pour suivre le stock de ses 5 articles principaux. On modélise le stock par deux tableaux de taille 5 :

- `noms[5]` : chaînes de caractères (exemples : "Riz", "Huile", "Savon", "Pates", "Lait");
- `qt[5]` : entiers (quantités en stock).

Les indices des tableaux vont de 0 à 4.

1. Initialisation (2 pts). Écris une procédure `InitialiserStock()` qui remplit `noms` avec les 5 articles ci-dessus et met toutes les quantités `qt` à 0.
2. Affichage (2 pts). Écris une procédure `AfficherStock()` qui affiche proprement le contenu du stock, ligne par ligne (exemple : 0 : Riz -- 50 kg).
3. Vente (3 pts). Écris une fonction `Vendre(indice : Entier, qteVendue : Entier) : Booleen` qui :
 - vérifie que `indice` est valide (compris entre 0 et 4) et que `qteVendue > 0`;
 - vérifie que le stock est suffisant (`qt[indice] >= qteVendue`);
 - si tout est correct, diminue le stock et retourne `VRAI`; sinon affiche un message d'erreur adapté et retourne `FAUX`.
4. Algorithme principal (3 pts). Écris l'algorithme principal qui :
 - appelle `InitialiserStock()`;
 - affiche le stock initial;
 - demande à l'utilisateur un indice d'article et une quantité à vendre;
 - appelle `Vendre` et affiche « Vente enregistrée » ou « Échec de la vente » selon la valeur retournée;
 - affiche le stock final.
5. Analyse (2 pts). Pourquoi est-il préférable d'utiliser une fonction retournant un booléen pour `Vendre` plutôt qu'une simple procédure? Quel avantage de la modularité cela illustre-t-il?

EXERCICE 14 Sujet type BEPC : mini-jeu « Pierre–Feuille–Ciseaux » — 10 pts

On veut programmer le jeu classique contre l'ordinateur. Règles : la Pierre bat les Ciseaux, les Ciseaux battent la Feuille, la Feuille bat la Pierre. Si les deux choix sont identiques, il y a match nul.

1. Choix de l'ordinateur (2 pts). Écris une fonction `ChoixOrdi()` : `Entier` qui retourne un entier aléatoire entre 1 et 3 (1 = Pierre, 2 = Feuille, 3 = Ciseaux). Indication : on dispose de la fonction `ALEA(1, 3)` qui fournit un entier au hasard entre 1 et 3.
2. Comparaison (4 pts). Écris une fonction `Resultat(joueur : Entier, ordi : Entier) : Chaine` qui retourne "Gagne", "Perdu" ou "Nul" selon les règles du jeu. Utilise une structure `SI ... ALORS ... SINON` claire (éventuellement avec des `SI` imbriqués).
3. Boucle de jeu (2 pts). Écris l'algorithme principal qui :
 - initialise `scoreJoueur` et `scoreOrdi` à 0 ;
 - répète (boucle `TANT QUE`) : demande le choix du joueur (1, 2 ou 3), appelle `ChoixOrdi`, appelle `Resultat`, met à jour les scores et affiche le résultat du tour ;
 - la boucle s'arrête dès que l'un des deux scores atteint 3 ;
 - affiche le grand gagnant final.
4. Amélioration (2 pts). Propose une modification pour gérer le cas où l'utilisateur saisit une valeur invalide (par exemple 5 ou -1) sans fausser le jeu. Décris la logique en français ou en pseudo-code (inutile de réécrire tout le programme).

EXERCICE 15 Sujet type BEPC : analyse d'un algorithme de tri simple — 8 pts

On donne l'algorithme suivant qui trie un tableau `T` de `N` entiers par ordre croissant (tri par sélection) :

```
Algorithme TriSelection
Variable
  T : Tableau[0..N-1] de Entier
  N, i, j, minIdx, temp : Entier
Debut
  // On suppose N et le tableau T déjà saisis
  Pour i de 0 à N-2 faire
    minIdx <- i
    Pour j de i+1 à N-1 faire
      Si T[j] < T[minIdx] Alors
        minIdx <- j
    Fin Si
  Fin Pour
  Si minIdx <> i Alors
    temp <- T[i]
    T[i] <- T[minIdx]
    T[minIdx] <- temp
```

```
    Fin Si
  Fin Pour
Afficher("Tableau trie")
Fin
```

1. Exécution pas à pas (3 pts). Soit $N = 5$ et $T = [4, 2, 5, 1, 3]$. Complète le tableau de trace ci-dessous pour les deux premières itérations de la boucle externe ($i = 0$ puis $i = 1$). Indique la valeur finale de `minIdx`, l'échange éventuellement effectué (indices concernés) et l'état du tableau T après l'itération.

i	minIdx final	Échange ? (indices)	T après l'itération
0			
1			

2. Nombre de comparaisons (2 pts). Combien de comparaisons $T[j] < T[\text{minIdx}]$ sont effectuées au total pour un tableau de taille N ? Donne la formule sous forme d'une somme en fonction de N , puis sa forme simplifiée.
3. Propriété de la boucle (2 pts). Énonce en français clair la propriété vérifiée au début de chaque itération i de la boucle externe : que peut-on dire de la partie gauche du tableau (indices 0 à $i - 1$) ?
4. Correction (1 pt). Si l'on remplace la condition `Si minIdx <> i Alors` par `Si T[minIdx] < T[i] Alors`, l'algorithme reste-t-il correct? Justifie ta réponse.

10 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : Cycle de vie et vocabulaire

1. B. Dans le cycle de vie en V, la phase de conception détaillée (choix des algorithmes et des structures de données) précède immédiatement le codage : on ne programme que ce qui a été conçu. Le recueil des besoins (A) est la toute première phase ; les tests (C) et la maintenance (D) viennent après le codage.
2. B. Afficher envoie un résultat vers la sortie standard, c'est-à-dire l'écran. Stocker une valeur (A) correspond à l'affectation `<-` ; lire au clavier (C) correspond à Saisir.
3. C. La boucle TANT QUE condition FAIRE ... FIN TANT QUE répète le bloc tant que la condition reste vraie (le nombre de répétitions n'est pas connu à l'avance). La boucle POUR (B) s'utilise quand le nombre d'itérations est connu ; SI (A) et SELON (D) sont des structures conditionnelles, pas des répétitions.
4. C. Une fonction retourne une valeur (avec Retourner) utilisable dans une expression, par exemple `m <- Max(a, b)`. Une procédure effectue des actions mais ne renvoie rien.

Réponses : 1-B, 2-B, 3-C, 4-C.

CORRIGÉ Exercice 2 — Vrai ou Faux ? Justifie

1. Faux. L'algorithme est une suite ordonnée et finie d'opérations décrite en pseudo-code, indépendante de tout langage ; le code source en C ou en Python est la traduction (l'implémentation) de cet algorithme dans un langage de programmation.
2. Faux. Une variable de type Entier ne peut stocker que des nombres sans partie décimale ; la valeur 12,5 exige le type Reel.
3. Faux. Les tests unitaires vérifient isolément chaque fonction ou procédure ; c'est la phase de validation (ou recette) qui vérifie que le logiciel complet répond au cahier des charges du client.
4. Vrai. La boucle POUR fait prendre au compteur `i` toutes les valeurs entières de la borne initiale (1) à la borne finale (10), en augmentant de 1 à chaque tour.
5. Faux. Le débogage consiste à rechercher et corriger les erreurs (bogues) d'un programme, souvent à l'aide d'un débogueur ; la rédaction de la documentation utilisateur relève de la phase de documentation, qui est distincte.

CORRIGÉ Exercice 3 — Définitions et correspondance

- 1-c : une variable est une zone de mémoire nommée, d'un type défini, dont la valeur peut changer au cours de l'exécution.
- 2-a : la compilation est la phase qui transforme le code source en programme exécutable (langage machine).
- 3-b : un algorithme est une suite ordonnée et finie d'opérations permettant de résoudre un problème.
- 4-d : un test unitaire est la vérification isolée du bon fonctionnement d'une fonction ou d'une procédure.
- 5-e : le cahier des charges est le document qui décrit précisément ce que le logiciel doit faire.

Réponses : 1-c, 2-a, 3-b, 4-d, 5-e.

CORRIGÉ Exercice 4 — Lecture d'algorithme : calcul d'une somme

1. Pour $n = 5$, la variable `somme` est initialisée à 0, puis la boucle ajoute successivement les valeurs de i :

$i = 1$: `somme` = $0 + 1 = 1$

$i = 2$: `somme` = $1 + 2 = 3$

$i = 3$: `somme` = $3 + 3 = 6$

$i = 4$: `somme` = $6 + 4 = 10$

$i = 5$: `somme` = $10 + 5 = 15$

La valeur finale de `somme` est donc 15 (on vérifie : $1 + 2 + 3 + 4 + 5 = 15$).

2. La structure de contrôle utilisée est la boucle `POUR` (boucle itérative à compteur), car le nombre de répétitions (n) est connu avant d'entrer dans la boucle.
3. Si l'utilisateur saisit 0, la boucle n'est pas exécutée et `somme` conserve sa valeur initiale 0. L'affichage est : La somme des 0 premiers entiers est : 0. Le programme reste correct grâce à l'initialisation `somme <- 0` avant la boucle.

CORRIGÉ Exercice 5 — Compléter un organigramme : maximum de trois nombres

On suit l'indication : on initialise `max` avec A , puis on le compare à B puis à C .

- Premier rectangle vide (après la saisie) : `max <- A`;

- Premier losange : $B > \text{max}$? — si Oui, rectangle : $\text{max} \leftarrow B$; si Non, on passe directement au test suivant;
- Deuxième losange : $C > \text{max}$? — si Oui, rectangle : $\text{max} \leftarrow C$; si Non, on passe directement à l’affichage;
- Ensuite : Afficher max , puis Fin.

L’algorithme équivalent en pseudo-code est :

```
Algorithme Maximum3
Variable
  A, B, C, max : Reel
Debut
  Saisir(A, B, C)
  max ← A
  Si B > max Alors
    max ← B
  Fin Si
  Si C > max Alors
    max ← C
  Fin Si
  Afficher("Le maximum est : ", max)
Fin
```

Vérification avec $A = 7, B = 12, C = 5$: max vaut 7, puis 12 (car $12 > 7$), et reste 12 (car $5 < 12$).
Affichage : 12, correct.

CORRIGÉ Exercice 6 — Écriture de fonction : année bissextile

```
Fonction EstBissextile(annee : Entier) : Booleen
Debut
  Si (annee MOD 400 = 0) Alors
    Retourner VRAI
  Sinon
    Si (annee MOD 4 = 0) ET (annee MOD 100 <> 0) Alors
      Retourner VRAI
    Sinon
      Retourner FAUX
    Fin Si
  Fin Si
Fin
```

Justification. On teste d’abord le cas exceptionnel : une année divisible par 400 est toujours bissextile (ex. 2000). Sinon, il faut être divisible par 4 sans être divisible par 100 (ex. 2024 est bissextile ; 1900 ne l’est pas, car $1900 \bmod 100 = 0$). Tous les autres cas retournent FAUX.

Jeux de vérification : $\text{EstBissextile}(2024) = \text{VRAI}$; $\text{EstBissextile}(1900) = \text{FAUX}$;
 $\text{EstBissextile}(2000) = \text{VRAI}$; $\text{EstBissextile}(2023) = \text{FAUX}$.

CORRIGÉ Exercice 7 — Conception de jeux de tests : décision d'admission

N°	note	Résultat attendu	Type de test	Justification
1	12	Admis	Cas normal	Valeur courante au-dessus du seuil : vérifie le comportement général.
2	10	Admis	Valeur limite basse	Le seuil exact : la condition est ≥ 10 , donc 10 doit donner « Admis ». C'est le test le plus important (erreur classique : écrire $>$ au lieu de $\geq$).
3	9,99	Recale	Valeur limite haute	Juste en dessous du seuil : vérifie que la frontière est bien placée.
4	-3	Recale (ou message d'erreur)	Valeur invalide	Une note négative est impossible : le programme ne doit pas « admettre » une telle valeur ; idéalement il devrait la rejeter.

Points de vigilance. Un bon jeu de tests explore toujours les valeurs limites (autour du seuil 10) et pas seulement des cas « évidents ». On peut ajouter 20 (note maximale) comme valeur limite haute du domaine valide, et 25 comme valeur invalide supérieure.

CORRIGÉ Exercice 8 — Algorithme complet : moyenne de la classe

```

Algorithme MoyenneClasse
Variable
  N, i : Entier
  note, somme, moyenne : Reel
Debut
  Afficher("Nombre d'élèves : ")
  Saisir(N)
  somme ← 0

```

```
Pour i de 1 a N faire
  Afficher("Note de l'eleve ", i, " : ")
  Saisir(note)
  somme <- somme + note
Fin Pour
moyenne <- somme / N
Afficher("La moyenne de la classe est : ", moyenne, " / 20")
Fin
```

Points de vigilance.

- **somme** est déclarée en Reel (les notes peuvent être décimales) et initialisée à 0 avant la boucle : sans cela, le résultat serait imprévisible.
- La division `somme / N` est placée après la boucle, jamais à l'intérieur.
- **N** étant supposé strictement positif (énoncé), il n'y a pas de division par zéro ; dans un programme réel, on vérifierait `N > 0` avant de calculer.

Vérification : pour $N = 3$ et les notes 10, 12, 14 : $\text{somme} = 36$, $\text{moyenne} = 36/3 = 12$. Correct.

CORRIGÉ Exercice 9 — Sujet type BEPC : gestion d'un stock de boutique

1. Initialisation (2 pts).

```
Procédure InitialiserStock()
Variable
  i : Entier
Debut
  noms[0] <- "Riz"
  noms[1] <- "Huile"
  noms[2] <- "Savon"
  noms[3] <- "Pates"
  noms[4] <- "Lait"
  Pour i de 0 a 4 faire
    qt[i] <- 0
  Fin Pour
Fin
```

2. Affichage (2 pts).

```
Procédure AfficherStock()
Variable
  i : Entier
Debut
  Pour i de 0 a 4 faire
    Afficher(i, " : ", noms[i], " -- ", qt[i], " kg")
  Fin Pour
Fin
```

3. Vente (3 pts).

```
Fonction Vendre(indice : Entier, qteVendue : Entier) : Booleen
```

```

Debut
  Si (indice < 0) OU (indice > 4) Alors
    Afficher("Erreur : indice invalide")
    Retourner FAUX
  Fin Si
  Si qteVendue <= 0 Alors
    Afficher("Erreur : quantite invalide")
    Retourner FAUX
  Fin Si
  Si qt[indice] < qteVendue Alors
    Afficher("Erreur : stock insuffisant")
    Retourner FAUX
  Fin Si
  qt[indice] <- qt[indice] - qteVendue
  Retourner VRAI
Fin

```

4. Algorithme principal (3 pts).

```

Algorithme GestionStock
Variable
  noms : Tableau[0..4] de Chaîne
  qt : Tableau[0..4] de Entier
  indice, qte : Entier
  ok : Booleen
Debut
  InitialiserStock()
  Afficher("--- Stock initial ---")
  AfficherStock()
  Afficher("Indice de l'article (0 à 4) : ")
  Saisir(indice)
  Afficher("Quantite a vendre : ")
  Saisir(qte)
  ok <- Vendre(indice, qte)
  Si ok = VRAI Alors
    Afficher("Vente enregistrée")
  Sinon
    Afficher("Echec de la vente")
  Fin Si
  Afficher("--- Stock final ---")
  AfficherStock()
Fin

```

5. Analyse (2 pts). Une fonction retournant un booléen permet au programme appelant de connaître le résultat de l'opération et de décider quoi faire ensuite (afficher « Vente enregistrée » ou « Échec ») : la décision est réutilisable et testable indépendamment. Une simple procédure effectuerait l'action « en silence » sans informer l'appelant. Cela illustre l'avantage de la modularité : chaque fonction a un rôle précis, peut être testée isolément (test unitaire) et réutilisée dans d'autres parties du programme.

Barème indicatif : 1. 2 pts (affectations 1 pt, boucle de mise à zéro 1 pt) ; 2. 2 pts ; 3. 3 pts (1 pt par vérification, retour booléen compris) ; 4. 3 pts (appels et enchaînement corrects) ; 5. 2 pts (rôle du booléen 1 pt, modularité 1 pt).

Points de vigilance. Les indices vont de 0 à 4 (pas de 1 à 5) ; les tableaux sont des variables globales accessibles dans les procédures ; la modification du stock ne doit avoir

lieu qu'après toutes les vérifications.

CORRIGÉ Exercice 10 — Sujet type BEPC : mini-jeu « Pierre-Feuille-Ciseaux »

1. Choix de l'ordinateur (2 pts).

```
Fonction ChoixOrdi() : Entier
Debut
    Retourner ALEA(1, 3)
Fin
```

2. Comparaison (4 pts).

```
Fonction Resultat(joueur : Entier, ordi : Entier) : Chaine
Debut
    Si joueur = ordi Alors
        Retourner "Nul"
    Sinon
        Si (joueur = 1 ET ordi = 3) OU (joueur = 2 ET ordi = 1) OU (joueur = 3 ET ordi = 2) Alors
            Retourner "Gagne"
        Sinon
            Retourner "Perdu"
        Fin Si
    Fin Si
Fin
```

Justification. Le joueur gagne dans exactement trois cas : Pierre (1) contre Ciseaux (3), Feuille (2) contre Pierre (1), Ciseaux (3) contre Feuille (2). L'égalité étant traitée avant, tous les cas restants sont perdus.

3. Boucle de jeu (2 pts).

```
Algorithme PierreFeuilleCiseaux
Variable
    joueur, ordi, scoreJoueur, scoreOrdi : Entier
    res : Chaine
Debut
    scoreJoueur <- 0
    scoreOrdi <- 0
    Tant Que (scoreJoueur < 3) ET (scoreOrdi < 3) Faire
        Afficher("Votre choix (1=Pierre, 2=Feuille, 3=Ciseaux) : ")
        Saisir(joueur)
        ordi <- ChoixOrdi()
        res <- Resultat(joueur, ordi)
        Si res = "Gagne" Alors
            scoreJoueur <- scoreJoueur + 1
        Fin Si
        Si res = "Perdu" Alors
            scoreOrdi <- scoreOrdi + 1
        Fin Si
        Afficher("Tour : ", res, " -- Score : ", scoreJoueur, " / ", scoreOrdi)
    Fin Tant Que
    Si scoreJoueur = 3 Alors
```

```

    Afficher("Vous êtes le grand gagnant !")
Sinon
    Afficher("L'ordinateur gagne la partie.")
Fin Si
Fin

```

Vérification de la condition d'arrêt : la boucle continue tant qu'aucun des deux scores n'a atteint 3 ; dès que l'un atteint 3, la condition $(\text{scoreJoueur} < 3)$ ET $(\text{scoreOrdi} < 3)$ devient fausse et la boucle s'arrête.

4. Amélioration (2 pts). On contrôle la saisie avec une boucle **TANT QUE** : tant que la valeur saisie n'est pas comprise entre 1 et 3, on affiche un message d'erreur et on redemande le choix. Ainsi, une valeur invalide (5, -1) ne déclenche ni tour de jeu ni modification des scores.

```

Saisir(joueur)
Tant Que (joueur < 1) OU (joueur > 3) Faire
    Afficher("Choix invalide, recommencez : ")
    Saisir(joueur)
Fin Tant Que

```

Barème indicatif : 1. 2 pts ; 2. 4 pts (cas nul 1 pt, trois cas gagnants 2 pts, cas perdus 1 pt) ; 3. 2 pts (condition d'arrêt 1 pt, mise à jour des scores 1 pt) ; 4. 2 pts.

Points de vigilance. Bien initialiser les scores avant la boucle ; ne pas oublier que le match nul ne modifie aucun score ; la condition du **TANT QUE** utilise ET (les deux scores doivent être < 3 pour continuer).

CORRIGÉ Exercice 11 — Sujet type BEPC : analyse d'un algorithme de tri simple

1. Exécution pas à pas (3 pts). Tableau initial : $T = [4, 2, 5, 1, 3]$.

Itération $i = 0$: on cherche le minimum dans $T[0..4]$. Les valeurs sont 4, 2, 5, 1, 3 ; le minimum est 1, à l'indice 3, donc $\text{minIdx} = 3$. Comme $\text{minIdx} \neq i$, on échange $T[0]$ et $T[3]$: $T = [1, 2, 5, 4, 3]$.

Itération $i = 1$: on cherche le minimum dans $T[1..4]$, c'est-à-dire parmi 2, 5, 4, 3. Le minimum est 2, déjà à l'indice 1, donc $\text{minIdx} = 1$. Comme $\text{minIdx} = i$, aucun échange : $T = [1, 2, 5, 4, 3]$.

i	minIdx final	Échange ? (indices)	T après l'itération
0	3	Oui : indices 0 et 3	[1, 2, 5, 4, 3]
1	1	Non ($\text{minIdx} = i$)	[1, 2, 5, 4, 3]

2. Nombre de comparaisons (2 pts). Pour chaque valeur de i , la boucle interne effectue une comparaison pour chaque j de $i + 1$ à $N - 1$, soit $(N - 1) - (i + 1) + 1 = N - 1 - i$ comparaisons. Au total :

$$\sum_{i=0}^{N-2} (N - 1 - i) = (N - 1) + (N - 2) + \dots + 1 = \frac{N(N - 1)}{2}.$$

Vérification avec $N = 5$: $\frac{5 \times 4}{2} = 10$ comparaisons ($4 + 3 + 2 + 1 = 10$). Correct.

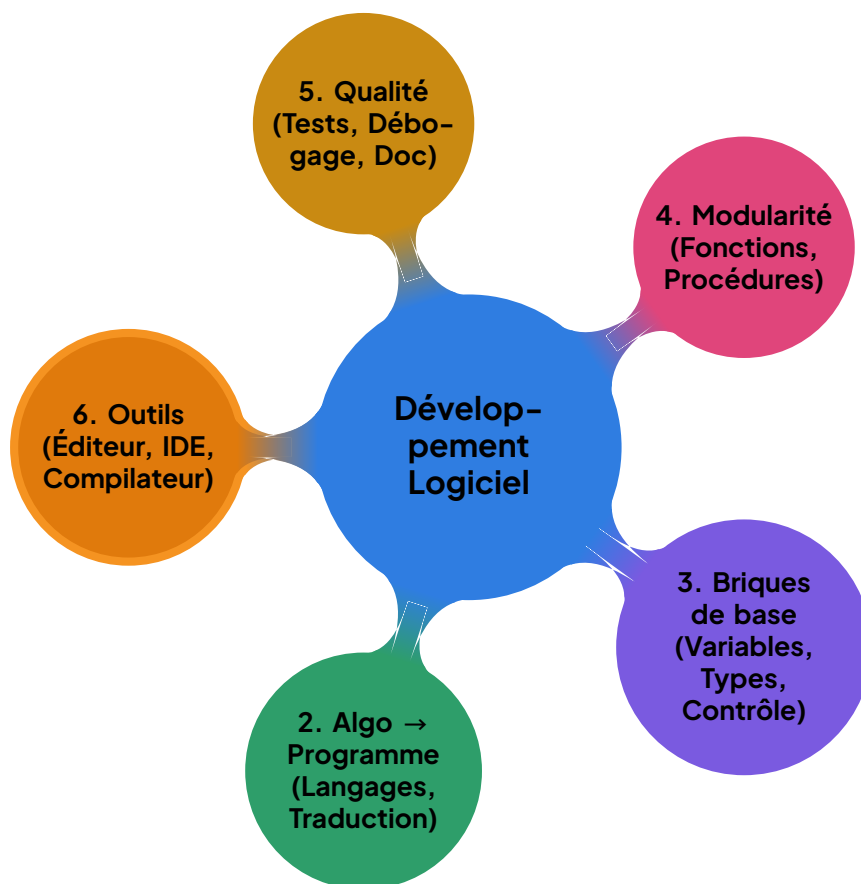
3. Propriété de la boucle (2 pts). Au début de chaque itération i , la partie gauche du tableau (indices 0 à $i - 1$) contient les i plus petits éléments du tableau, déjà triés par ordre croissant et placés à leur position définitive. C'est l'invariant de boucle : chaque itération agrandit cette zone triée d'un élément.

4. Correction (1 pt). Oui, l'algorithme reste correct. Par construction, $T[\text{minIdx}]$ est le minimum de la zone $T[i..N-1]$, donc on a toujours $T[\text{minIdx}] \leq T[i]$. Deux cas : si $\text{minIdx} = i$, alors $T[\text{minIdx}] = T[i]$, la condition $T[\text{minIdx}] < T[i]$ est fausse et l'on n'échange pas (comme avant); si $\text{minIdx} \neq i$, alors $T[\text{minIdx}] < T[i]$ et l'échange a lieu (comme avant). Le comportement est donc identique.

Barème indicatif : 1. 3 pts (1,5 pt par ligne : minIdx , échange, état du tableau); 2. 2 pts (somme 1 pt, forme simplifiée 1 pt); 3. 2 pts; 4. 1 pt.

Points de vigilance. La boucle externe s'arrête à $N - 2$ (le dernier élément est automatiquement à sa place); la recherche du minimum porte sur la zone non triée (de i à $N - 1$); dans la formule, ne pas oublier que le nombre de termes de la somme est $N - 1$.

11 FICHE BILAN



Carte mentale : Concepts de base du développement logiciel (Leçon 15)

FICHE BILAN L'essentiel à connaître pour l'examen

11.1 Définitions clés

- **Logiciel** : Ensemble de programmes, procédures, règles et documentation relatifs au fonctionnement d'un système de traitement de l'information.
- **Algorithme** : Suite finie et non ambiguë d'instructions (opérations élémentaires) permettant de résoudre un problème ou d'obtenir un résultat. Écrit en langage naturel ou pseudo-code.
- **Programme** : Traduction d'un algorithme dans un **langage de programmation** (C, Python, JavaScript, etc.) compréhensible par la machine après traduction.
- **Code source** : Texte du programme écrit par le développeur (lisible par l'humain).

- **Code objet / Exécutable** : Résultat de la traduction (binaire), directement exécutable par le processeur.
- **Variable** : Zone mémoire nommée, identifiée par un **type** (entier, réel, chaîne, booléen), dont la valeur peut changer pendant l'exécution.
- **Structure de contrôle** : Instruction qui détermine l'ordre d'exécution des autres instructions (Séquence, Sélection Si, Itération Pour/TantQue).
- **Fonction** : Sous-programme qui effectue un traitement et renvoie une valeur (un seul résultat).
- **Procédure** : Sous-programme qui effectue un traitement sans renvoyer de valeur (effet de bord : affichage, modification de variables globales/paramètres).
- **Bogue (Bug)** : Erreur dans le code source provoquant un comportement inattendu.
Débogage : Processus de localisation et de correction des bogues.

11.2 Repères essentiels (à connaître par cœur)

Concept	Description / Règle	Exemple / Note
Cycle de vie (en V ou cascade)	1. Analyse des besoins 2. Conception (Algo/MCD) 3. Codage (Programmation) 4. Tests (Unitaires, Intégration, Validation) 5. Déploiement & Maintenance	Itératif dans la pratique moderne (Agile).
Traduction	Compilateur : Traduit tout le code source en exécutable avant exécution (C, C++). Rapide à l'exécution. Interpréteur : Traduit et exécute instruction par instruction (Python, JS, BASIC). Plus lent, mais interactif.	Erreur de compilation = syntaxe. Erreur à l'exécution = logique/exception.
Types de base	Entier (int), Réel (float/double), Caractère (char), Chaîne (string), Booléen (bool : Vrai/Faux)	Typage fort (C) vs faible/dynamique (Python, JS). En 3ème : on précise le type en pseudo-code.
Structures de contrôle	Séquence : Instructions les unes après les autres. Sélection : Si ... Alors ... Sinon ... FinSi (choix). Itération : Pour (nb tours connu) / TantQue (condition).	Éviter les boucles infinies (condition de sortie modifiée dans le corps).
Portée des variables	Locale : Déclarée dans un bloc (fonction/-procédure), invisible dehors. Globale : Déclarée hors blocs, visible partout (à limiter).	Privilégier le passage par paramètres (entrées/-sorties).
Test / Qualité	Jeu de tests : Cas normaux, limites (bords), invalides/erroneux.	Un logiciel non testé est un logiciel qui ne marche pas.

Traces d'exécution : Tableau (Variable | Valeur) pour suivre l'algo.

Documentation : Commentaires (//, /* */), guide utilisateur, doc

11.3 Méthodes express

MÉTHODE Écrire un algorithme structuré (Pseudo-code)

1. Analyser : Identifier les données d'entrée (Données), le résultat attendu (Résultats) et les traitements (Traitements).
2. Déclarer les variables (Nom : Type) dans la section Variables.
3. Écrire le corps (Début ... Fin) en utilisant uniquement les 3 structures de contrôle (Séquence, Si, Pour/TantQue).
4. Modulariser : Si une tâche est répétée ou complexe → Créer une Fonction (re-tourne valeur) ou Procédure (action).
5. Tester sur papier avec un jeu de tests (tableau de traces) avant de coder.

MÉTHODE Déboguer efficacement (Premier niveau)

1. Lire le message d'erreur (ligne, type : syntaxe, exécution, logique).
2. Utiliser des Afficher (prints) stratégiques pour suivre les valeurs des variables (trace pas à pas).
3. Isoler le bloc fautif (commenter des parties, tester des fonctions seules).
4. Corriger une seule chose à la fois, recompiler, retester.

À RETENIR Checklist avant le Bac

- ☐ Je sais définir : Logiciel, Algorithme, Programme, Variable, Type, Fonction, Procédure, Bogue.
- ☐ Je cite les 5 grandes phases du cycle de vie d'un logiciel (Analyse → Maintenance).
- ☐ Je distingue Compilateur (ex : C) et Interpréteur (ex : Python, JS) et leurs impacts.
- ☐ Je connais les 3 structures de contrôle fondamentales : Séquence, Sélection (Si), Itération (Pour, TantQue).
- ☐ Je sais déclarer une variable avec un type adapté (Entier, Réel, Chaîne, Booléen).
- ☐ Je sais écrire un algorithme complet en pseudo-code (Variables, Début, Fin, indentation correcte).

- ☐ Je sais construire un jeu de tests pertinent (cas normaux, limites, erreurs).
- ☐ Je sais réaliser une table de traces (exécution pas à pas) pour valider un algorithme.
- ☐ Je comprends l'intérêt de la modularité (Fonctions/Procédures) : lisibilité, réutilisation, maintenance.
- ☐ Je sais expliquer la différence entre paramètre entrée (valeur) et sortie (référence/adresse) – notion de passage par valeur/adresse.
- ☐ Je connais les bases de la documentation : commentaires code, nommage clair (camelCase/snake_case), guide utilisateur.
- ☐ Je suis capable de proposer une correction simple face à un bogue classique (boucle infinie, décalage d'indice, condition inversée).

**Signé • OnBuch+**Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC**OnBuch+**